Back to the Drawing Board: A Critical Evaluation of Few-Step Solvers for Production Diffusion Models

Jianrong Lu, Zhiyu Zhu, Hui Liu, and Junhui Hou

***Abstract*—While recent works have indicated that training-free few-step sampling solvers can achieve nearly quality-lossless acceleration for diffusion and flow-matching models, their real impact on production diffusion generation is not fully understood. In this work, we develop a comprehensive systemization for training-free few-step sampling on diffusion models by enumerating the quality metrics that the literature and industry actually use to judge sampling acceleration.**

We present a critical analysis of training-free few-step sampling under practical, production diffusion environments by carefully characterizing the realistic quality evaluation dimensions a service cares about—human-preference alignment, aesthetic quality, dense instruction following, and in-image long-text rendering—which the standard FID/CLIP protocol overlooks. Our findings are rather surprising: contrary to the established belief that few-step solvers are virtually quality-lossless, we show that they remain near-lossless only on simple distribution-alignment metrics yet degrade substantially under the metrics a production service actually cares about, and thus deliver no effective few-step acceleration. We go even further and propose a novel, state-of-the-art parallel sampling method, *ParaLin*++, built on our conference-accepted ParaLin, and show via an extensive set of experiments across Stable Diffusion-v2, LSUN Church, FLUX.1-dev, and Ming-Flash-Omni 2.0 that it preserves the full-step Euler quality profile at $3.6\times$–$5.2\times$ (up to $71.4\times$ at 1000 steps), $1.68\times$ on FLUX and $1.21\times$ on Ming. Project page: https://parasolver-turbo.github.io/.

***Index Terms*—Diffusion Probabilistic Models, Parallel Sampling, Generative Models, Model Acceleration, Ordinary Differential Equations, System of Linear Equations.**

I. Introduction

DIFFUSION probabilistic models (DPMs) [20], [50] are now a backbone of industrial image and video generation, powering production text-to-image services built on large flow-matching models such as FLUX [3] and Ming-Flash-Omni 2.0 [1], with applications from high-fidelity synthesis [42], [4] to molecular generation [55], [40] and a broad range of generative tasks [56], [28], [33], [10], [34], [57], [26]. In deployment, sampling cost is the dominant inference bottleneck: each high-resolution sample requires a long sequence of neural function evaluations (NFEs), and the latency of this *sequential* trajectory constrains serving throughput, inference cost, and on-device feasibility.

The speed gap of diffusion sampling. The reference discretization of a DPM is a fixed N-step ODE/SDE integrator, typically Euler with $N{=}30$ steps for flow-matching models. Accelerating this pipeline has followed two structurally different routes, alongside a broader landscape of training-free speedups [15], [60], [16], [12], [29], [37]. The first route *shortens the discretization*: high-order or few-step training-free solvers such as DDIM [46], DPMSolver [31], [32], and UniPC [59], together with distillation/consistency approaches [43], [47], [25], [27], cut the step count at the cost of changing the discretized trajectory. The second route *removes solver latency instead*: parallel sampling refines many time points concurrently, beginning with Picard iteration in ParaDiGMS [44] and progressing through triangular [51] and banded [35] nonlinear solvers, recently made NFE-efficient by the partial-linear formulation of ParaLin [36]. These two routes buy speed from different places: the first spends *fewer integration steps*, the second spends *parallel compute to erase the solver's temporal dependency*.

A taxonomy of what acceleration claims rest on. The decisive axis of a training-free accelerator is the *metric* that certifies it "quality-lossless," not the speed lever it uses. The literature has converged on distributional and text-alignment proxies—FID measures closeness of the generated *distribution* to a reference, CLIP measures coarse prompt–image embedding similarity—while production text-to-image services are post-trained and judged on a deeper profile: semantic composition (GenEval [14]), dense instruction following (DPG-Bench [21]), and long in-image text rendering (LongTextBench [13]). A step-shortening strategy that looks lossless under FID can still alter the prompt-conditioned trajectory and sit in the same cell as a production failure, because it changes the discrete trajectory on which prompt and text fidelity depend even when the marginal image distribution is preserved. We therefore focus on *step-parity* parallelization: keep the complete fixed-step system, but solve its coupled integration equations in parallel so that latency is removed without changing the discretization on which production-grade

This project was supported in part by the NSFC Excellent Young Scientists Fund 62422118, in part by the Hong Kong RGC under Grants RFS2627-1S02, R1018-25F, N_CityU1114/25, and 11219324, and in part by the Hong Kong UGC under Grant UGC/FDS11/E03/24 and Grant UGC/FDS11/E03/25.

J. Lu and J. Hou are with the Department of Computer Science, City University of Hong Kong, Hong Kong SAR, China. (email:jrong.alvin@gmail.com; jh.hou@cityu.edu.hk)

Z. Zhu is with City University of Hong Kong (Dongguan), Dongguan, China, and also with the Department of Computer Science, City University of Hong Kong, Hong Kong SAR, China. (e-mail: zhiyuzhu2@my.cityu.edu.hk;)

H. Liu is with the School of Computing and Information Sciences, Saint Francis University, Hong Kong SAR, China (e-mail: h2liu@sfu.edu.hk).

prompt and text fidelity depends.

Our contributions. We make the following contributions.

I. Systemization of training-free diffusion acceleration. We start by establishing a comprehensive systemization of the quality metrics used to evaluate training-free sampling acceleration for diffusion and flow-matching models. Table I enumerates the evaluation protocols of a broad set of samplers—DDPM, DDIM, DPM-Solver, DPMSolver++, DPM-Solver-v3, UniPC, distillation and consistency methods, ParaDiGMS, ParaSolver, ParaLin, and ours—and shows that the literature almost universally certifies acceleration only with FID/CLIP-class distributional and text-alignment metrics. Such metrics measure statistical alignment of the marginal distribution, not whether a generated sample is well composed, instruction-following, or text-legible, and are largely outdated with respect to the human-preference, aesthetic, instruction-following, and long-text-rendering axes that industry reports. We believe that prior few-step sampling works have neglected the crucial quality requirements of production diffusion generation systems on the axes relevant to deployment. To the best of our knowledge, our work is the first to consider production diffusion generation environments and add an industrial-grade audit (GenEval, DPG-Bench, LongTextBench) alongside FID. As a result, our evaluations lead to conclusions that contradict common beliefs in the literature, e.g., we show that production diffusion generation even with the classical DPMSolver++ suffers large quality loss and cannot achieve any effective acceleration: on FLUX, a 16-step DPMSolver++ configuration that is FID-indistinguishable from the 30-step Euler reference already loses 21%–24% GenEval, 6.5–8.2 DPG-Bench points, and 67%–70% LongTextBench (Table VI).

II. A stronger step-parity parallel solver, ParaLin++. We build the strongest available realization of the step-parity route to test how far this route reaches. Starting from ParaLin's ϱ-nonlinear partial-linear system, we add an adaptive nonlinearization iteration, a successive-over-relaxation (SOR) update for both nonlinear and linear blocks, a sliding window, and an adaptive stopping rule based on relative historical errors and an exponential moving average. We prove that the SOR update is stable for $0 < \omega < 2/(1+L)$ (contraction factor $L < 1$) and that the system admits the sequential trajectory as its unique convergent root.

III. Industrial-grade quality–speed audit in practice. We run the audit across Stable Diffusion-v2, LSUN Church, FLUX.1-dev, Ming-Flash-Omni 2.0, five CFG values, three industrial benchmarks, visual comparisons, and ablations. Our key lessons are:

1) *FID-lossless is not production-lossless.* On FLUX, a 1.8× few-step claim that is valid under FID (gap $\leqslant$ 0.03) coincides with a 21%–24% GenEval drop, a 6.5–8.2 DPG-Bench point drop, and a 67%–70% LongTextBench drop—an operational failure the standard evaluation hides.
2) *Step-parity keeps the reference profile.* Keeping all 30 steps and parallel-solving gives 1.68× on FLUX and 1.21× on Ming-Flash-Omni 2.0 with the Euler-level industrial profile (0.64 vs. 0.66 GenEval, 84.24 vs. 83.97 DPG, 0.26 vs. 0.27 LTB), and 3.6×–5.2× (up to 71.4× at 1000 steps) on Stable Diffusion-v2/LSUN Church.
3) *Guidance sharpens the failure.* Across CFG 0–14, the completed DPG-Bench matrix shows ParaLin++ tracking Euler closely (82.18–84.24 vs. 82.25–83.97) while 20-step UniPC/DPMSolver++ drop to 68.85–79.67.
4) *Report iterations and industrial metrics, not only FID.* A deployable speedup claim should separate discrete steps from solver iterations/NFE and be measured against the same-CFG 30-step Euler reference on an industrial metric; an FID-only claim is not actionable.

Implications of our study. Because a step-shortening route can be simultaneously FID-lossless and production-broken, FID cannot certify training-free speedup at the fidelity that production demands. This argues for reporting an industrial-quality profile alongside FID and for treating step-parity as the safer route when prompt and text fidelity are part of the service-level objective.

The remainder of this paper is organized as follows. Section II provides the necessary background and reviews existing training-free accelerators. Section III systematizes the quality metrics used to evaluate training-free acceleration and shows that FID-lossless claims collapse under production-quality metrics. Section IV first reviews ParaLin's partial-linear reformulation and then designs the stronger parallel solver ParaLin++ (Section IV-B). Section V reports the industrial-grade quality–speed audit and ablations. Section VI concludes.

II. Background

Sequential Integral Solvers. The forward diffusion process degrades a clean image $\mathbf{X}_T$ into random noise $\mathbf{X}_0$ according to the stochastic differential equation (SDE)

$$d\mathbf{X}_t = f(t)\mathbf{X}_t\,dt + g(t)\,d\mathbf{W}, \tag{1}$$

where $d\mathbf{W}$ is the standard Wiener process. This induces a marginal distribution $\mathbf{X}_t \sim \mathcal{N}(\alpha_t\mathbf{X}_T, \sigma_t\mathbf{I})$ with noise schedules α_t and σ_t. Generation reverses the process through the reverse-time SDE

$$d\mathbf{X}_t = \underbrace{\left[f(t)\mathbf{X}_t - g^2(t)\nabla_{\mathbf{X}_t}\log p(\mathbf{X}_t)\right]}_{\varphi(\mathbf{X}_t,t)}dt + \underbrace{g(t)}_{\sigma_t}\,d\mathbf{W}, \tag{2}$$

where a neural network $\mathbf{S}_\theta(\cdot)$ approximates the score. Let $\Phi(t, s, \mathbf{X}_s)$ denote the integral solution of Eq. (2) over $[s, t]$:

$$\Phi(t, s, \mathbf{X}_s) = \mathbf{X}_s + \int_s^t \varphi(\mathbf{X}_\tau, \tau)\,d\tau + \int_s^t \sigma_\tau\,d\mathbf{W}. \tag{3}$$

Parallel Integral Solvers. ParaSolver [35] provides a unified framework for sequential and parallel sampling. Given a trajectory divided into N sub-intervals $0 = t_0 < t_1 < \cdots < t_N = T$, it forms the banded nonlinear system

$$\begin{cases} \hat{\mathbf{X}}_{t_0} - \mathbf{X}_{t_0} = \mathbf{0}, \\ \hat{\mathbf{X}}_{t_{n+1}} - \Phi(t_{n+1}, t_n, \hat{\mathbf{X}}_{t_n}) = \mathbf{0}, \quad n \in \{0, 1, \ldots, N-1\}. \end{cases} \tag{4}$$

Starting from an initial estimate, the concatenated state is updated by

$$\hat{\mathbf{X}}^{(k+1)}_{t_0:t_N} = \hat{\mathbf{X}}^{(k)}_{t_0:t_N} - \mathcal{G}^{(k)}\mathcal{R}^{(k)}_{t_0:t_N}, \tag{5}$$

where $\mathcal{R}^{(k)}_{t_0} = \mathbf{0}$ and $\mathcal{R}^{(k)}_{t_{n+1}} = \hat{\mathbf{X}}^{(k)}_{t_{n+1}} - \Phi(t_{n+1}, t_n, \hat{\mathbf{X}}^{(k)}_{t_n})$. The solver matrix $\mathcal{G}^{(k)}$ selects the algorithm: $\mathcal{G}^{(k)} = \mathbf{I}$ gives fixed-point iteration [6], while $\mathcal{G}^{(k)} = (\mathcal{J}^{(k)})^{-1}$ gives Newton–Raphson iteration [23]. ParaLin [36] reduces the NFE burden of these fully nonlinear alternatives by exploiting the partial linearity of the integral solver.

A. Existing Training-Free Accelerators

Parallel sampling accelerates DPMs by iteratively refining multiple points along ODE/SDE trajectories. ParaDiGMS [44] first introduced this direction through Picard–Lindelöf fixed-point iteration. ParaTAA [51] formulates sampling as a triangular nonlinear system, and ParaSolver [35] improves efficiency with a banded system and hierarchical parallelization. These methods are training-free, but their NFE overhead limits wall-clock benefits, especially on a single GPU. ParaLin [36] exploits the partial-linear structure of diffusion integration to reduce this overhead and generalizes these fully nonlinear formulations as special cases.

Beyond parallelization, other acceleration directions include model or sample partitioning [39], [38], [54], [24], [30], guidance acceleration [53], trajectory stitching [41], nested diffusion [11], minimum-step prediction [58], Picard consistency models [45], and parallel-sampling theory [17], [8], [62]. ParaLin++ is complementary to these methods: it accelerates a training-free parallel diffusion solver and can, in principle, be combined with several of these systems.

III. Systemization of Training-Free Diffusion Acceleration

We systematize training-free diffusion acceleration through the quality metrics on which its "quality-lossless" claims rest, rather than through the speed lever it uses, because a sampler certified by a weak metric is not deployment-ready. We first enumerate the evaluation protocols of the academic acceleration literature (Table I) and identify what these methods actually measure; we then test the headline few-step quality claims under an industrial-quality audit and find them to fail (Section V, summarized in Table VI).

A. Quality Metrics in the Training-Free Acceleration Literature

Table I surveys the quality metrics used to evaluate a broad set of training-free samplers and their close relatives, from DDPM and DDIM through high-order solvers (DPM-Solver, DPMSolver++, DPM-Solver-v3, UniPC) and distillation/consistency methods, to parallel samplers (ParaDiGMS, ParaTAA, ParaSolver, ParaLin) and our ParaLin++. Two patterns are immediate. *First*, nearly every method is evaluated with distributional and text-alignment metrics only: FID (and its variants sFID, clean-FID) measures closeness of the generated *distribution* to a reference distribution, while CLIP score/CLIP-FID measures coarse prompt–image embedding similarity, and a handful additionally report Inception-based diversity (IS, Precision–Recall). These quantify *statistical alignment*—whether the marginal image distribution or the prompt–image embedding stays close to a reference—not whether a specific generated sample succeeds at production tasks such as semantic composition, dense instruction following, or long-text rendering. *Second*, none of these methods is audited on the production-level quality axes that industrial generators report: human-preference alignment, aesthetic quality, semantic composition (GenEval [14]), dense instruction following (DPG-Bench [21]), and in-image long-text rendering (LongTextBench [13]). The metric set that has certified training-free samplers for nearly a decade is largely outdated with respect to industry practice: production text-to-image services such as FLUX [3] and Ming-Flash-Omni 2.0 [1] report GenEval-, DPG-Bench-, and LongTextBench-style numbers, whereas FID/CLIP are scarce or absent in their evaluation reports. A sampler certified only by the academic protocol can therefore look lossless yet be unusable on production prompts.

B. An Industrial-Quality Audit of Training-Free Accelerators

The metrics gap is not cosmetic. We further test the same training-free accelerators under the industrial-quality profile—GenEval, DPG-Bench, and LongTextBench—on the production-grade FLUX.1-dev [3], against the same-CFG 30-step Euler reference; the result is summarized in Table VI and elaborated in Section V. Methods advertised as nearly quality-lossless under FID collapse on the production metrics: 16-step UniPC and DPMSolver++ keep FID within 0.03 of the 30-step Euler reference (which would formally support a $1.8\times$ "lossless" speedup), yet GenEval falls from 0.66 to 0.50–0.52, DPG-Bench from 83.97 to 75.75–77.46, and LongTextBench from 0.27 to 0.08–0.09. An image distribution can stay FID-plausible while the model omits requested objects, breaks spatial relations, or renders unreadable text. *Hence, under the metrics a production service actually cares about, the standard few-step solvers can no longer guarantee near-lossless quality and therefore cannot achieve any effective acceleration at few steps.*

This failure is structural rather than incidental. Step-shortening solvers (DDIM, DPMSolver++, UniPC) buy fewer discrete steps by altering the prompt-conditioned trajectory, and the altered trajectory is what breaks dense-prompt and text fidelity; FID merely fails to reflect this break because it overlooks sample-level semantics. Parallel sampling instead keeps all N discrete steps and removes only the solver's temporal dependency, preserving the production trajectory and the quality that depends on it. We call this the *step-parity* route, and ParaLin++ (Section IV-B) is designed as its strongest realization. The rest of the paper extends the industrial-quality audit (Section V) and strengthens the step-parity solver (Sections IV–IV-B).

TABLE I
QUALITY METRICS USED TO EVALUATE TRAINING-FREE DIFFUSION/FLOW-MATCHING ACCELERATION IN THE LITERATURE AND IN THIS WORK. "DIST." = DISTRIBUTIONAL, REFERENCE-BASED METRICS OF THE FID CLASS (FID, SFID, CLEAN-FID, IS, PRECISION–RECALL); "T2I-ALIGN." = TEXT–IMAGE ALIGNMENT METRICS (CLIP SCORE, CLIP-FID); "INDUSTRIAL" = PRODUCTION GENERATION-QUALITY BENCHMARKS (GENEVAL, DPG-BENCH, LONGTEXTBENCH). THE ACADEMIC ACCELERATION LITERATURE STOPS AT DISTRIBUTIONAL/TEXT-ALIGNMENT PROXIES AND OMITS THE INDUSTRIAL-QUALITY AXES THAT PRODUCTION SERVICES RELY ON; TO THE BEST OF OUR KNOWLEDGE ONLY THIS WORK ADDS AN INDUSTRIAL-GRADE AUDIT. ✓/✗: METRIC REPORTED / NOT REPORTED BY THE METHOD'S OWN EVALUATION.

Method	Dist.	T2I-align.	Industrial
DDPM [20]	✓	✗	✗
DDIM [46]	✓	✗	✗
DPM-Solver [31]	✓	✗	✗
DPMSolver++ [32]	✓	✓	✗
DPM-Solver-v3 [61]	✓	✗	✗
UniPC [59]	✓	✓	✗
Euler (flow reference)	✓	✗	✗
Prog. distillation [43]	✓	✓	✗
Consistency model [48]	✓	✗	✗
InstaFlow [29]	✓	✓	✗
ParaDiGMS [44]	✓	✗	✗
ParaTAA [51]	✓	✗	✗
ParaSolver [35]	✓	✓	✗
ParaLin [36]	✓	✓	✗
ParaLin++ (ours)	✓	✓	✓

TABLE II
PRACTICAL ERROR BOUNDS FOR DIFFUSION FRAMEWORKS IN THE EDM DESIGN SPACE. PARAMETERS FOLLOW TABLE 1 OF [22], WITH $\beta_{\min} = 0.1$, $\beta_d = 19.9$, AND $\|\mathbf{S}_{\theta*}(\mathbf{X}_t, t) - \epsilon_t\| < \zeta$ FOR A SMALL STEP SIZE $h = s - t < 1$. ROWS GIVE THE SCHEDULE SETTINGS, THE σ_t/α_t SUBSTITUTION, AND THE RESULTING BOUND; COLUMNS ARE THE DIFFUSION FRAMEWORKS.

	EDM [22]	VE [50]	VP [50]
Schedule settings	$\alpha_t = 1$ $\sigma_t = t$	$\alpha_t = 1$ $\sigma_t = \sqrt{t}$	$\alpha_t = 1/\sqrt{e^{\frac{1}{2}\beta_d t^2 + \beta_{\min} t}}$ $\sigma_t = \alpha_t \sqrt{e^{\frac{1}{2}\beta_d t^2 + \beta_{\min} t} - 1}$
Substitution $\sigma_t/\alpha_t =$	t	$\sqrt{t}$	$\sqrt{e^{\frac{1}{2}\beta_d t^2 + \beta_{\min} t} - 1}$
Final error bound	$\|\mathcal{Z}_s\| \leqslant \zeta(s-t) = \zeta h$	$\|\mathcal{Z}_s\| \leqslant \zeta(\sqrt{s} - \sqrt{t})$	$\|\mathcal{Z}_s\| \leqslant \zeta\alpha_t(\sigma_s/\alpha_s - \sigma_t/\alpha_t)$

IV. EXPLORING THE SPACE: FROM PARALIN TO PARALIN++

A. The ParaLin Framework

Parallel solvers reformulate sequential diffusion integration into the nonlinear system in Eq. (4) and Eq. (5), but denoising many states in every iteration causes large NFE overhead. The accepted ParaLin framework addresses this by reducing fully nonlinear score computation to partially linear operations. This section briefly recalls that formulation, which serves as the foundation of ParaLin++.

1) Reformulation of the Sequential Integral Solver: Existing approaches typically develop parallel solvers based on the sequential integral formulation in Eq. (3), which results in a fully nonlinear system of equations in Eq. (4) when employing the nonlinear score function $\mathbf{S}_\theta(\cdot)$. We reveal that the sequential integral solver in Eq. (3) can be expressed in a more elementary form and reduced to a linear solver, as formally stated in the following Proposition 1.

Proposition 1 (Linear approximation (Proof in App. A))**.** *The sequential integral solver* $\Phi(t, s, \mathbf{X}_s)$ *from* Eq. (3) *can be decomposed into a linear solver* $\mathcal{H}$ *and a negligible error term* $\mathcal{Z}_s$*:*

$$\Phi(t, s, \mathbf{X}_s) = \mathcal{H}\left(t, s, \mathbf{X}_s, \mathbf{X}_T(\mathbf{X}_s)\right) + \mathcal{Z}_s, \tag{6}$$

where the linear sequential integral solver $\mathcal{H}(\cdot)$ *is defined as:*

$$\mathcal{H}(\cdot) = \frac{\alpha_t}{\alpha_s}\mathbf{X}_s - \alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda} \frac{\mathbf{X}_\lambda - \alpha_\lambda \mathbf{X}_T(\mathbf{X}_s)}{\sigma_\lambda} d\lambda + \int_{\lambda_s}^{\lambda_t} g(\lambda) d\mathbf{W}, \tag{7}$$

Here, $h = s - t$ *denotes the step size and* $\lambda_t = \log(\alpha_t/\sigma_t)$ *. Under the assumption that the model prediction error is bounded by* ζ*, the decomposition error satisfies* $\|\mathcal{Z}_s\| \leqslant \zeta\alpha_t(\frac{\sigma_s}{\alpha_s} - \frac{\sigma_t}{\alpha_t})$*, which is negligible for* $h \leqslant 1$*.*

Remark 1. *The small step size* h *is well-founded in practice. Theoretically, convergence guarantees for diffusion solvers strictly require bounded steps (*$h \leqslant 1$ *[9]), while practically high-fidelity generation involves more fine discretization (e.g.,* $N \geqslant 5$ *steps), ensuring* $h = 1/N \leqslant 0.2$ *remains small to render the error negligible. As detailed in Tab. II, this error term is small for practical discretization. Specifically, for the EDM framework, the bound simplifies to* $\|\mathcal{Z}_s\| \leqslant \zeta h$*; for the VE and VP schedules, the corresponding bounds shrink to first order as* $h = s - t$ *becomes small. This ensures that the approximation error remains negligible under high-fidelity sampling.*

Now, following ParaSolver [35], the linear sequential integral solver in Eq. (7) can be decomposed into many integral functions on different time subintervals, written as

$$\mathbf{X}_{t_{n+1}} = \mathcal{H}(t_{n+1}, t_n, \mathbf{X}_{t_n}, \mathbf{X}_{t_N}), n \in \{0, 1, \cdots, N-1\}. \tag{8}$$

Motivated by recent parallel algorithms [35], [49], this inherently sequential problem can be addressed in a parallel framework. Specifically, such a sequence of cascaded linear functions can be interpreted as a system of fully linear equations. Denote by $\{\mathbf{X}_{t_n}, n = 0, \cdots, N\}$ a set of points exactly on the diffusion trajectory, and $\left\{\hat{\mathbf{X}}_{t_n}, n = 0, \cdots, N\right\}$ the unknown variables to be optimized towards the trajectory. The cascade of integral formulations as Eq. (8) can then be reformulated as the solutions to the following *fully linear equations*.

Definition 1 (Fully Linear Equation System)**.** *We define the system of fully linear equations for the sequential sampling process in* Eq. (8) *as*

$$\begin{cases} \hat{\mathbf{X}}_{t_0} - \mathbf{X}_{t_0} = \mathbf{0}, \\ \hat{\mathbf{X}}_{t_{n+1}} - \mathcal{H}(t_{n+1}, t_n, \hat{\mathbf{X}}_{t_n}, \hat{\mathbf{X}}_{t_N}) = \mathbf{0}, \quad n \in \{0, 1, \cdots, N-1\}. \end{cases} \tag{9}$$

The complete linearity of the above system permits exceptionally efficient solving with negligible computational cost.

However, this very full linearity prevents them from achieving the sample quality of the sequential integral solvers in Eq. (3), as they lack information regarding the target data distribution. To bridge this quality gap, we introduce the target distribution information into the linear system by supplying the estimated clean sample $\hat{\mathbf{X}}_{t_N}$. Recall that in the forward process of DPMs, we can compute a clean sample predicted $\hat{\mathbf{X}}_{t_N}$ as

$$\hat{\mathbf{X}}_{t_N}(\hat{\mathbf{X}}_{t_n}, \theta) = \frac{\hat{\mathbf{X}}_{t_n} - \sigma_{t_n}\mathbf{S}_\theta(\hat{\mathbf{X}}_{t_n}, t_n)}{\alpha_{t_n}}, \tag{10}$$

where $\mathbf{X}_{t_n} \sim \mathcal{N}(\alpha_{t_n}\mathbf{X}_{t_N}, \sigma_{t_n}\mathbf{I})$ where α and σ are the noise schedule. Note that the diffusion model consumes most computational resources on the score-matching process, i.e., estimating corresponding noise or clean samples by a neural network. In this paper, to lessen such burden of score matching, we explore the consistency of reverse diffusion process and propose to divide the NEs to a partial-linear equation system, namely ϱ-nonlinear equation system. Specifically, due to the diffusion models make progressive transition from pure noise to clean data distribution, the score function is highly unstable on the high noise region, which is also evidenced by recent research [41], [39]. For compactness, let $\mathcal{P}_i^{(j)} = \hat{\mathbf{X}}_{t_N}(\hat{\mathbf{X}}_{t_i}^{(j)}, \theta)$ denote the clean-sample prediction formed from state i at iteration j, and write $\mathcal{H}_i(x, y) = \mathcal{H}(t_{i+1}, t_i, x, y)$. Based on such an insight, to further balance the score estimation accuracy and computational consumption, we divide the original NEs shown as Eq. 4 into ϱ-Nonlinear Equation System in Definition 2, where for the unstable and stable score region, we apply the nonlinear and linear solvers, respectively.

Definition 2 (ϱ-Nonlinear Equation System)**.** *We define a ϱ-nonlinear equation system for the sequential sampling process in* Eq. (8) *as*

$$\begin{cases} \hat{\mathbf{X}}_{t_0} - \mathbf{X}_{t_0} = \mathbf{0}, \\ \hat{\mathbf{X}}_{t_{n+1}} - \mathcal{H}\Big(t_{n+1}, t_n, \hat{\mathbf{X}}_{t_n}, \hat{\mathbf{X}}_{t_N}(\hat{\mathbf{X}}_{t_n}, \theta)\Big) = \mathbf{0}, n \in [0, \varrho - 1], \\ \hat{\mathbf{X}}_{t_{n+1}} - \mathcal{H}\Big(t_{n+1}, t_n, \hat{\mathbf{X}}_{t_n}, \hat{\mathbf{X}}_{t_N}(\hat{\mathbf{X}}_{t_{\varrho-1}}, \theta)\Big) = \mathbf{0}, n \in [\varrho, N - 1]. \end{cases} \tag{11}$$

Remark 2. *The scalar ϱ serves as a critical transition point between nonlinear (score) and linear (fixed score) regimes in the system. The first ϱ equations evolve into computationally costly nonlinear equations, reducing into exiting nonlinear parallel system in* Eq. (4)*. Subsequent equations (for $n \geqslant \varrho$) retain computational efficiency as linear equations, as they depend only on the fixed prediction from the $(\varrho - 1)^{th}$ time step.*

Remark 3. *The above framework establishes a unified paradigm, subsuming all existing parallel methods such as ParaDiGMS, ParaTAA, and ParaSolver as special cases. In particular, existing methods are fully nonlinear equation systems, corresponding to the limiting case $\varrho = N - 1$.*

Proposition 2 (Accumulated Error Bound (Proof in App. B))**.** *For any ϱ, let $\{\hat{\mathbf{X}}_{t_n}\}_{n=0}^N$ be the solution to the ϱ-nonlinear surrogate system in* Eq. (11) *and $\{\mathbf{X}_{t_n}\}_{n=0}^N$ the exact sequential integral trajectory, with deviation $\mathbf{\Delta}_{t_n} = \hat{\mathbf{X}}_{t_n} - \mathbf{X}_{t_n}$ and clean-sample estimation error $\boldsymbol{\xi}_n = \mathbf{E}_{t_n} - \mathbf{X}_T$. Under the high-quality sampling regime $h \leqslant 1$, the deviation obeys the per-step recurrence*

$$\|\mathbf{\Delta}_{t_{n+1}}\| \leqslant |\mathcal{A}_n|\|\mathbf{\Delta}_{t_n}\| + |\mathcal{B}_n|\|\boldsymbol{\xi}_n\| + \|\mathcal{Z}_n\|, \tag{12}$$

with $\mathcal{A}_n = \frac{\alpha_{t_{n+1}}}{\alpha_{t_n}}(1 - \Delta\lambda_n)$ and $\mathcal{B}_n = \alpha_{t_{n+1}}\Delta\lambda_n\left(1 - \frac{\Delta\lambda_n}{2}\right)$. Starting from $\mathbf{\Delta}_{t_0} = \mathbf{0}$ and using $|\mathcal{A}_n| \leqslant 1$, $|\mathcal{B}_n| = \mathcal{O}(h)$, and $\|\mathcal{Z}_n\| = \zeta\,\mathcal{O}(h)$ (Proposition 1), unrolling over the N steps yields the accumulated error bound

$$\begin{aligned} \|\mathbf{\Delta}_{t_N}\| &\leqslant \sum_{n=0}^{N-1}\Big(\prod_{j=n+1}^{N-1}|\mathcal{A}_j|\Big)\big(|\mathcal{B}_n|\|\boldsymbol{\xi}_n\| + \|\mathcal{Z}_n\|\big) \\ &\leqslant \sum_{n=0}^{N-1}\big(|\mathcal{B}_n|\|\boldsymbol{\xi}_n\| + \|\mathcal{Z}_n\|\big) \leqslant N\,\mathcal{O}(h)\big(\|\boldsymbol{\xi}\|_{\max} + \zeta\big). \end{aligned} \tag{13}$$

The bound is structurally independent of ϱ: ϱ only changes the source of the estimator $\mathbf{E}_{t_n}$ (and hence $\|\boldsymbol{\xi}_n\|$), not the coefficients $\mathcal{A}_n$ and $\mathcal{B}_n$.

Remark 4. *Prop. 2 guarantees that different ϱ does not impact the quality of the generated samples. Our experiments also confirm this, showing that varying ϱ does not degrade the FID and CLIP scores of the generated sample across various diffusion processes like variance preserving (VP) and variance exploding (VE). We note that ϱ is less of a hyperparameter and more of a hardware configuration parameter. For good efficiency, ϱ can simply be set to available processor counts.*

B. Designing ParaLin++: A Stronger Step-Parity Solver

ParaLin++ accelerates the ParaLin solver from two complementary perspectives. First, it retains ParaLin's adaptive nonlinearization and immediate update strategy, which exploit the latest refined states within an iteration. Second, it introduces SOR and a relative adaptive stopping rule to reduce the remaining iterations. The sliding-window and initialization strategies are then integrated into the complete algorithm.

1) Adaptive Nonlinearization Iteration: We investigate the update rule in Eq. (5) to solve the prescribed ϱ-nonlinear system. Specializing Eq. (5) to our system yields the following iterative scheme:

$$\hat{\mathbf{X}}_{t_0:t_N}^{(k+1)} = \hat{\mathbf{X}}_{t_0:t_N}^{(k)} - \mathcal{G}^{(k)}\mathcal{R}_{t_0:t_N}^{(k)}, \tag{14}$$

where the residual term $\mathcal{R}_{t_{n+1}}^{(k)}$ with $\mathcal{R}_{t_0}^{(k)} = \hat{\mathbf{X}}_{t_0}^{(k)} - \mathbf{X}_{t_0} = 0$ is defined as:

$$\mathcal{R}_{t_{n+1}}^{(k)} = \begin{cases} \hat{\mathbf{X}}_{t_{n+1}}^{(k)} - \mathcal{H}_n\big(\hat{\mathbf{X}}_{t_n}^{(k)}, \mathcal{P}_n^{(k)}\big), & n \in [0, \varrho - 1], \\ \hat{\mathbf{X}}_{t_{n+1}}^{(k)} - \mathcal{H}_n\big(\hat{\mathbf{X}}_{t_n}^{(k)}, \mathcal{P}_{\varrho-1}^{(k)}\big), & n \in [\varrho, N - 1]. \end{cases} \tag{15}$$

Observe that this update rule applies a Jacobi-type iteration, where all components $\mathbf{X}_{t_1:t_N}^{(k+1)}$ are updated exclusively from the old components $\mathbf{X}_{t_1:t_N}^{(k)}$, resulting in delayed information propagation across time steps. Inspired by the classical Gauss-Seidel iteration, we propose a more efficient immediate update strategy wherein newly updated components are incorporated immediately into subsequent components' updating within the same iteration. Specifically, we first update the nonlinear-region components $\mathbf{X}_{t_n}^{(k)}$ for $n \in \{1, \cdots, \varrho\}$. Upon obtaining

the refined nonlinear component $\mathbf{X}_{t_\varrho}^{(k+1)}$, we instantly propagate this refinement forward to compute the remaining linear-region components (i.e., $\mathbf{X}_{t_{\varrho+1}}^{(k+1)}, \dots, \mathbf{X}_{t_N}^{(k+1)}$). This approach ensures faster convergence by propagating the latest information forward, leading to the following improved update rule:

$$\hat{\mathbf{X}}_{t_0:t_N}^{(k+1)} = \hat{\mathbf{X}}_{t_0:t_N}^{(k)} - \mathcal{G}^{(k)}\mathcal{R}_{t_0:t_N}^{(k)}, \tag{16}$$

where the residual term $\mathcal{R}_{t_{n+1}}^{(k)}$ now incorporates the latest refined components:

$$\mathcal{R}_{t_{n+1}}^{(k)} = \begin{cases} \hat{\mathbf{X}}_{t_{n+1}}^{(k)} - \mathcal{H}_n\big(\hat{\mathbf{X}}_{t_n}^{(k)}, \mathcal{P}_n^{(k)}\big), & n \in [0, \varrho-1], \\ \hat{\mathbf{X}}_{t_{n+1}}^{(k)} - \mathcal{H}_n\big(\hat{\mathbf{X}}_{t_n}^{(k+1)}, \mathcal{P}_{\varrho-1}^{(k)}\big), & n \in [\varrho, N-1]. \end{cases} \tag{17}$$

We observe that once the nonlinear components converge, the predicted clean sample $\hat{\mathbf{X}}_{t_N}(\hat{\mathbf{X}}_{t_{\varrho-1}}^{(k)}, \theta)$ becomes unchanged, implying that no further precise information can be propagated for the linear equations in subsequential iterations. To address this, we progressively convert the linear equations into nonlinear ones, such that the number of nonlinear equations remains unchanged as the iteration proceeds. Formally, at iteration k, suppose the unconverged components are $\hat{\mathbf{X}}_{t_n}^{(k)}, \cdots, \hat{\mathbf{X}}_{t_N}^{(k)}$, we maintain nonlinearity in the residuals $\{\mathcal{R}_{t_{i+1}}^{(k)}, i = n, n+1, \cdots, n+\varrho-1\}$ of the first ϱ components through the following formulation:

$$\hat{\mathbf{X}}_{t_n:t_N}^{(k+1)} = \hat{\mathbf{X}}_{t_n:t_N}^{(k)} - \mathcal{G}^{(k)}\mathcal{R}_{t_n:t_N}^{(k)}, \tag{18}$$

where the residual term $\mathcal{R}_{t_{i+1}}^{(k)}$ can adaptively convert the linear equations into non-linear ones:

$$\mathcal{R}_{t_{i+1}}^{(k)} = \begin{cases} \hat{\mathbf{X}}_{t_{i+1}}^{(k)} - \mathcal{H}_i\big(\hat{\mathbf{X}}_{t_i}^{(k)}, \mathcal{P}_i^{(k)}\big), & i \in [n, n+\varrho-1], \\ \hat{\mathbf{X}}_{t_{i+1}}^{(k)} - \mathcal{H}_i\big(\hat{\mathbf{X}}_{t_i}^{(k+1)}, \mathcal{P}_{n+\varrho-1}^{(k)}\big), & i \in [n+\varrho, N-1]. \end{cases} \tag{19}$$

where $\mathcal{R}_{t_n}^{(k)} = \hat{\mathbf{X}}_{t_n}^{(k)} - \mathbf{X}_{t_n} = \mathbf{0}$. This approach ensures continuous refinement of $\hat{\mathbf{X}}_{t_N}(\hat{\mathbf{X}}_{t_{n+\varrho-1}}^{(k)}, \theta)$, yielding increasingly accurate solutions for the linear equations.

Proposition 3 (Convergence Analysis)**.** *The update rule in* Eq. (18) *converges, within $K \leqslant T$ iterations, to the* partially linear surrogate system*, i.e., the ϱ-nonlinear system defined in* Eq. (11)*. The surrogate trajectory $\{\hat{\mathbf{X}}_{t_n}\}_{n=0}^N$ differs from the exact sequential integral trajectory $\{\mathbf{X}_{t_n}\}_{n=0}^N$ by an accumulated error bounded in Proposition 2. In other words, the iteration closes the layer-3 to layer-2 gap (iterative solve → surrogate system), while the layer-2 to layer-1 gap (surrogate → exact sequential sampler) is the bounded deviation studied separately in Proposition 2.*

Proof. See Appendix C for the complete derivation. □

2) Convergence Acceleration via Successive Over-Relaxation: While the immediate update strategy enhances information flow within each iteration, the solver may still require a considerable number of iterations (K) to converge, particularly for high-dimensional problems. To further reduce the iteration count, we incorporate the Successive Over-Relaxation (SOR) method, a well-established technique for accelerating the convergence of iterative solvers. The core idea is to introduce a relaxation factor ω to extrapolate the update step, pushing the solution more aggressively towards the root.

The standard Gauss-Seidel-like update can be viewed as a two-step process. First, an intermediate state $\hat{\mathbf{Z}}^{(k+1)}$ is computed based on the residual:

$$\hat{\mathbf{Z}}^{(k+1)} = \hat{\mathbf{X}}^{(k)} - \mathcal{G}^{(k)}\mathcal{R}^{(k)}. \tag{20}$$

The SOR update then modifies this step by taking a weighted average of the current state $\hat{\mathbf{X}}^{(k)}$ and the intermediate state $\hat{\mathbf{Z}}^{(k+1)}$:

$$\hat{\mathbf{X}}^{(k+1)} = (1-\omega)\hat{\mathbf{X}}^{(k)} + \omega\hat{\mathbf{Z}}^{(k+1)}, \tag{21}$$

where ω is the relaxation factor. Setting $\omega = 1$ recovers the original update rule, while $\omega > 1$ (over-relaxation) typically accelerates convergence for the systems encountered in our problem. We now integrate this SOR mechanism into our solver for both the nonlinear and linear components.

Update for Nonlinear Components. For the first ϱ components within the current window, which are processed in parallel, we first compute their intermediate states based on the nonlinear residuals defined in Eq. (19):

$$\hat{\mathbf{Z}}_{t_{n+1}:t_{n+\varrho}}^{(k+1)} = \hat{\mathbf{X}}_{t_{n+1}:t_{n+\varrho}}^{(k)} - \mathcal{G}^{(k)}\mathcal{R}_{t_{n+1}:t_{n+\varrho}}^{(k)}. \tag{22}$$

Subsequently, we apply the SOR update to obtain the refined states for the nonlinear block:

$$\hat{\mathbf{X}}_{t_{n+1}:t_{n+\varrho}}^{(k+1)} = (1-\omega)\hat{\mathbf{X}}_{t_{n+1}:t_{n+\varrho}}^{(k)} + \omega\hat{\mathbf{Z}}_{t_{n+1}:t_{n+\varrho}}^{(k+1)}. \tag{23}$$

Update for Linear Components. For the remaining components ($i \in [n+\varrho, N-1]$), we combine the SOR method with the immediate update strategy. These components are updated sequentially, allowing the most recent information to propagate immediately. For each linear component, we compute the intermediate state:

$$\hat{\mathbf{Z}}_{t_{i+1}}^{(k+1)} = \hat{\mathbf{X}}_{t_{i+1}}^{(k)} - \mathcal{G}^{(k)}\mathcal{R}_{t_{i+1}}^{(k)}, \tag{24}$$

where the residual $\mathcal{R}_{t_{i+1}}^{(k)}$ is computed using the already updated value $\hat{\mathbf{X}}_{t_i}^{(k+1)}$, as specified in Eq. (19). The final updated state is then obtained via the SOR formulation:

$$\hat{\mathbf{X}}_{t_{i+1}}^{(k+1)} = (1-\omega)\hat{\mathbf{X}}_{t_{i+1}}^{(k)} + \omega\hat{\mathbf{Z}}_{t_{i+1}}^{(k+1)}. \tag{25}$$

By integrating SOR, our proposed solver, ParaLin++, significantly reduces the number of iterations K required to reach the desired tolerance, thereby providing a substantial improvement in overall sampling efficiency without compromising the quality of the generated samples.

Proposition 4 (Relaxation Factor Analysis)**.** *The successive over-relaxation update guarantees convergence if the relaxation factor ω is chosen within the range $0 < \omega < 2/(1+L)$, where $L < 1$ is the contraction factor of the underlying Gauss-Seidel-like iterative solver.*

Proof. See Appendix D for the complete derivation. □

Remark 5. *Empirically, selecting ω within the range of $(1.0, 1.08]$ effectively accelerates the process without compromising the generation quality.*

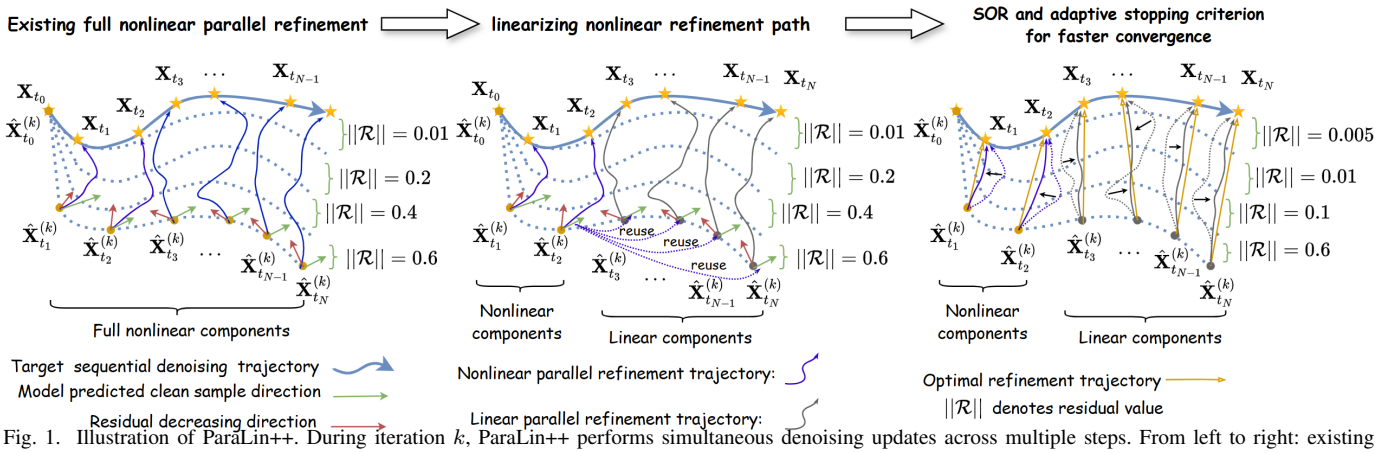

Fig. 1. Illustration of ParaLin++. During iteration k, ParaLin++ performs simultaneous denoising updates across multiple steps. From left to right: existing full nonlinear parallel methods, our partial linear method, and our method for faster convergence.

3) Sliding Window Strategy: To optimize computational efficiency, ParaSolver utilizes a sliding window of fixed size p, retaining only the most recent p denoised points $\{\hat{\mathbf{X}}_{t_n}^{(k)} : n = 0, \ldots, p-1\}$ for parallel processing. This approach enhances parallel performance while adhering to GPU memory limitations. Adapted to our framework, the update rule becomes:

$$\hat{\mathbf{X}}_{t_n:t_{n+p}}^{(k+1)} = \hat{\mathbf{X}}_{t_n:t_{n+p}}^{(k)} - \mathcal{G}^{(k)}\mathcal{R}_{t_n:t_{n+p}}^{(k)}, \tag{26}$$

where the residual term $\mathcal{R}_{t_{i+1}}^{(k)}$ over a sliding window becomes:

$$\mathcal{R}_{t_{i+1}}^{(k)} = \begin{cases} \hat{\mathbf{X}}_{t_{i+1}}^{(k)} - \mathcal{H}_i\big(\hat{\mathbf{X}}_{t_i}^{(k)}, \mathcal{P}_i^{(k)}\big), & i \in [n, n+\varrho-1], \\ \hat{\mathbf{X}}_{t_{i+1}}^{(k)} - \mathcal{H}_i\big(\hat{\mathbf{X}}_{t_i}^{(k+1)}, \mathcal{P}_{n+\varrho-1}^{(k)}\big), & i \in [n+\varrho, p-1]. \end{cases} \tag{27}$$

4) Adaptive Stopping Criterion: Prior works [44], [51], [35] typically employ a single tolerance, denoted as δ, to control the state difference across all timesteps. However, we find that this coarse-grained method cannot accurately identify states that have already converged, leading to superfluous iteration steps to update them. To address this inefficiency, we introduce a more fine-grained, adaptive stopping criterion.

Our approach is based on the principle of relative reduction to a historical average. Instead of comparing the current update error to a fixed threshold, we evaluate it against a fraction of the average historical error. This allows the criterion to dynamically adjust to the convergence progress.

Specifically, following [44], [51], [35], let the error of state $\hat{\mathbf{X}}_{t_i}$ at iteration $k \in [0, K]$ be defined as:

$$\mathcal{E}_{t_i}^{(k)} = \frac{1}{D\sigma_{t_i}^2} \left\| \hat{\mathbf{X}}_{t_i}^{(k+1)} - \hat{\mathbf{X}}_{t_i}^{(k)} \right\|_F^2,$$

where D is the dimensionality of the state $\hat{\mathbf{X}}_{t_i}$ and $\|\cdot\|_F$ denotes the Frobenius norm. We maintain a running average of all historical errors, $\bar{\mathcal{E}}_{t_i}$, up to the current iteration k:

$$\bar{\mathcal{E}}_{t_i} = \frac{1}{k+1} \sum_{j=0}^{k} \mathcal{E}_{t_i}^{(j)}.$$

The stopping condition is then met when the current error, $\mathcal{E}_{t_i}^{(k)}$, is significantly smaller than this historical error average, scaled by a tolerance δ:

$$\mathcal{E}_{t_i}^{(k)} \leqslant \delta \cdot \bar{\mathcal{E}}_{t_i}.$$

Furthermore, the tolerance δ is not a fixed hyperparameter but is itself updated adaptively to refine the stopping condition. We introduce an update mechanism with an EDM (Exponential Decay Mean) decay rate, β, which allows the tolerance to respond to the recent convergence behavior. The tolerance for the next step, δ, is updated as follows:

$$\delta = \beta\delta + (1-\beta) \cdot \mathcal{M}\Big(\{\mathcal{E}_{t_i}^{(k)} \mid i \in [n, n+p-1]\}\Big),$$

where $\mathcal{M}(\cdot)$ computes the median value which provides a robust estimate of recent error.

This dual-adaptive strategy, comparing to the historical average and dynamically updating the tolerance threshold, enables a more efficient convergence. We empirically find it terminates iterations more precisely when a state has stabilized, thereby reducing the number of steps required while preserving sample quality.

5) Initialization: **Initialization.** To establish the diffusion trajectory, we adopt the ParaSolver methodology by performing M steps of a sequential integral solver as preconditioning steps. The initialization process is formally expressed as:

$$\begin{cases} \hat{\mathbf{X}}_{t_n}^{(0)} = \mathbf{X}_{t_n}, & \text{if } 0 \leqslant n < M, \\ \hat{\mathbf{X}}_{t_n}^{(0)} = \mathcal{H}\Big(t_n, t_{n-1}, \hat{\mathbf{X}}_{t_{n-1}}^{(0)}, \hat{\mathbf{X}}_{t_N}(\hat{\mathbf{X}}_{t_{M-1}}^{(0)}, \theta)\Big), & \text{if } M \leqslant n < N, \end{cases} \tag{28}$$

where the initial state $\mathbf{X}_{t_0}$ is sampled from a standard Gaussian distribution, $\mathbf{X}_{t_0} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. When $M = 0$, following [44], all samples are initialized with random noise.

6) Complete Algorithm: Algorithm 1 and Figure 1 outlines the full procedure of the proposed method. Initially, an array of starting guesses $\{\hat{\mathbf{X}}_{t_n}^{(0)} : n = 0, \ldots, p\}$ is prepared either through random noises or inexpensive preconditioning steps (Line 1). The parallel sampling loop then begins at Line 3. In Lines 5–8, all nonlinear equations are solved simultaneously. Subsequently, Lines 9–12 compute the solutions to the linear equations, leveraging the predicted clean samples obtained from the nonlinear solving phase. This step is computationally efficient since it avoids model evaluations. The discrepancy between the newly computed states and the current states is assessed at Line 13, followed by determining the feasible window sliding stride at Line 15. Line 16 initializes s new points beyond the current window based on the computed

Algorithm 1: ParaLin++: a partial nonlinear parallel integral solver for diffusion models

Input : Diffusion model S_θ, noise schedule α_t and σ_t, subinterval number N, preconditioning steps M, tolerance δ, batch window size p, sample dimension D, nonlinear degree ϱ, EMA decay rate β, relaxation factor ω.

Output : A sample.

1 Initialize $\{\hat{\mathbf{X}}_{t_n}^{(0)}, n = 0, \cdots, p\}$ by Eq. (28) // Initialize with a few sampling steps.

2 Initialize $\{\mathcal{E}_{t_n}^{(0)} = 0, n = 0, \cdots, N\}$ // Initialize error list.

3 $n, k \leftarrow 0, 0$, $k \in [0, K]$, and $n \in [0, N-1]$

4 **while** $n < N$ **do**

5 $\hat{\mathbf{X}}_{t_N}(\hat{\mathbf{X}}_{t_i}^{(k)}, \theta) = \frac{\hat{\mathbf{X}}_{t_i}^{(k)} - \sigma_{t_i}\mathbf{s}_\theta(\hat{\mathbf{X}}_{t_i}^{(k)}, t_i)}{\alpha_{t_i}}, \forall i \in \{n, \cdots, n+\varrho-1\}$. // Predict clean samples in parallel.

6 $\mathcal{R}_{t_{i+1}}^{(k)} = \hat{\mathbf{X}}_{t_{i+1}}^{(k)} - \mathcal{H}\Big(t_{i+1}, t_i, \hat{\mathbf{X}}_{t_i}^{(k)}, \hat{\mathbf{X}}_{t_N}(\hat{\mathbf{X}}_{t_i}^{(k)}, \theta)\Big), \forall i \in \{n, \cdots, n+\varrho-1\}$ // Nonlinear residual.

7 $\hat{\mathbf{Z}}_{t_{n+1}:t_{n+\varrho}}^{(k+1)} = \hat{\mathbf{X}}_{t_{n+1}:t_{n+\varrho}}^{(k)} - \mathcal{G}^{(k)}\mathcal{R}_{t_{n+1}:t_{n+\varrho}}^{(k)}$ //Obtain intermediate states.

8 $\hat{\mathbf{X}}_{t_{n+1}:t_{n+\varrho}}^{(k+1)} = (1-\omega)\hat{\mathbf{X}}_{t_{n+1}:t_{n+\varrho}}^{(k)} + \omega\hat{\mathbf{Z}}_{t_{n+1}:t_{n+\varrho}}^{(k+1)}$ // Update the nonlinear components.

9 **for** $i \in \{n+\varrho, n+\varrho+1, \cdots, n+p-1\}$ **do**

10 $\mathcal{R}_{t_{i+1}}^{(k)} = \hat{\mathbf{X}}_{t_{i+1}}^{(k)} - \mathcal{H}\Big(t_{i+1}, t_i, \hat{\mathbf{X}}_{t_i}^{(k+1)}, \hat{\mathbf{X}}_{t_N}(\hat{\mathbf{X}}_{t_{n+\varrho-1}}^{(k)}, \theta)\Big)$ in Eq. (17) // Linear residual.

11 $\hat{\mathbf{Z}}_{t_{i+1}}^{(k+1)} = \hat{\mathbf{X}}_{t_{i+1}}^{(k)} - \mathcal{G}^{(k)}\mathcal{R}_{t_{i+1}}^{(k)}$ // Obtain intermediate states.

12 $\hat{\mathbf{X}}_{t_{i+1}}^{(k+1)} = (1-\omega)\hat{\mathbf{X}}_{t_{i+1}}^{(k)} + \omega\hat{\mathbf{Z}}_{t_{i+1}}^{(k+1)}$ // Update the linear components.

13 $\mathcal{E}_{t_{i+1}}^{(k)} = \frac{1}{D\sigma_{t_{i+1}}^2}||\mathbf{X}_{t_{i+1}}^{(k+1)} - \mathbf{X}_{t_{i+1}}^{(k)}||_F^2, \forall i \in \{n, \cdots, n+p-1\}$ // Compute error for each component.

14 $\bar{\mathcal{E}}_{t_{i+1}} = \frac{1}{1+k}\sum_{j=0}^{k}\mathcal{E}_{t_{i+1}}^{(j)}, \forall i \in \{n, \cdots, n+p-1\}$ // Compute average error for each component.

15 $s = \min_{i+1}\left(\{i+1 | \mathcal{E}_{t_{i+1}}^{(k)} > \delta^2\bar{\mathcal{E}}_{t_{i+1}} \forall i \in [n, n+p)\} \cup \{n+p\}\right)$ // The sliding stride.

16 $\hat{\mathbf{X}}_{t_{i+1}}^{(k+1)} = \mathcal{H}\Big(t_{i+1}, t_i, \hat{\mathbf{X}}_{t_i}^{(k+1)}, \hat{\mathbf{X}}_{t_N}(\hat{\mathbf{X}}_{t_{n+\varrho-1}}^{(k)}, \theta)\Big), \forall i \in \{n+p, \cdots, n+p+s-1\}$ // Initialize new components for the next window.

17 $e = \mathcal{M}\Big(\{\mathcal{E}_{t_{i+1}}^{(k)} | i = n, \cdots, n+p-1\}\Big)$ // Compute the median value of the errors.

18 $\delta = \beta\delta + (1-\beta)e$ // Adaptive tolerance updating.

19 $n \leftarrow n+s, \quad k \leftarrow k+1, \quad p \leftarrow \min(p, N-n)$ // Update the iteration indexs.

Return: $\hat{\mathbf{X}}_{t_N}^{(K)}$

stride. Finally, Lines 17–18 dynamically adjust the tolerance threshold δ based on the real-time residuals.

V. Industrial-Grade Quality–Speed Audit

A. Experiment Settings

Evaluation Metrics and Datasets. We report function evaluations (NFEs), iterations, wall-clock time, FID [19], and CLIP score [18]. For Stable Diffusion-v2 and LSUN Church, FID and CLIP are computed on 5000 random samples, using ImageNet-1k with classifier guidance $w = 7.5$ and the LSUN model, respectively. For FLUX, FID is computed on 5000 FluxPrompting [52] samples for the 12B model [3]. The CIFAR-10 less-step study uses 50,000 samples, as reported in Table V. For deployment-oriented evaluation, we additionally report GenEval [14], DPG-Bench [21], and LongTextBench [13], which assess semantic composition, dense instruction following, and long in-image text rendering, respectively.

Models. The evaluation covers latent-space Stable Diffusion-v2, CIFAR-10, the 12B FLUX flow-matching model, the pixel-space LSUN Church model, and VP/VE/EDM processes. The industrial-quality study centers on FLUX and uses the production-oriented Ming-Flash-Omni 2.0 generator [1] as an industrial reference system.

Algorithms. ParaLin++ is applied to DDPM [20], DDIM [46], and DPMSolver [31]. It is compared with ParaLin [36], ParaDiGMS [44], ParaSolver [35], and relevant few-step solvers.

Hyperparameters. Following [35], [51], we use the best tolerances reported for ParaDiGMS and ParaSolver. For ParaLin++, we set $\varrho = 8$ unless otherwise specified, use eight NVIDIA RTX 3090 GPUs for multi-GPU settings, and search δ, β, and ω independently for each sampler. Additional details are given in the supplementary material.

FLUX industrial-benchmark protocol. The FLUX.1-dev study uses 1024×1024 generation with guidance scale 3.5 on eight GPUs. The Euler sampler with 30 discrete diffusion steps is the reference. UniPC and DPM-Solver++ are evaluated with their standard sequential few-step samplers at $16, 20, 24, 26$, and 30 discrete diffusion steps. ParaLin++ keeps 30 discrete diffusion steps and uses the proposed parallel solver with tolerance $\delta = 0.01$, nonlinear-transition parameter $\varrho = 1$, parallel window size $p = 100$, and a solver-step cap of 1000. LongTextBench is the average of the English and Chinese text scores. Speedup is measured from the sampler/model forward-path time of the GenEval generation pass and excludes model loading and scoring; these timing measurements are used consistently across the detailed FLUX table.

Ming-Flash-Omni 2.0 industrial-benchmark protocol. The Ming-Flash-Omni 2.0 study uses the Ming-Flash-Omni 2.0 generator at 1024×1024 with guidance scale 5.0 on eight GPUs. Euler, UniPC, and DPM-Solver++ are evaluated with their standard sequential samplers at the listed step settings. ParaLin++ keeps 30 discrete diffusion steps and uses the proposed parallel solver with tolerance $\delta = 0.01$, nonlinear-transition parameter $\varrho = 1$, parallel window size $p = 100$, and a solver-step cap of 1000. LongTextBench uses 320 prompts with 4 images each, seed 42, hybrid OCR, a bounding-box score threshold of 0.85, and reports the average of the English and Chinese scores. DPG-Bench uses 1065 images with the mPLUG visual-question-answering model. GenEval uses 553 prompts with 4 images each, bf16 generation, and 8 evaluation workers. The table reports the complete 16-, 20-, 24-, 26-, and 30-step quality matrix. All speedup values use the CFG-5 GenEval generation pass against the 30-step Euler reference, excluding model loading and scoring.

B. Experimental Results

Latent-space Speedup Comparison. Table III shows the experimental results on DDPM, DDIM, DPMSolver, and their

TABLE III
COMPARISONS ON STABLE DIFFUSION-V2, WITH CLASSIFIER GUIDANCE W = 7.5. THE VISUAL COMPARISONS ARE SHOWN IN THE SUPPLEMENTARY FIGURES.

Steps	Method	Stable Diffusion-v2					
		Iters ↓	NFE ↓	CLIP↑	FID↓	Time (s)↓	Speedup↑
1000	DDPM [20]	1000	1000	25.6	55.9	128.0	1.0×
	DDPM + ParaDiGMS [44]	65	2024	25.6	55.6	32.8	3.9×
	DDPM + ParaSolver [35]	32	1065	25.6	55.3	15.9	8.1×
	DDPM + ParaLin [36]	32	195	25.6	55.7	3.2	40.0×
	DDPM + ParaLin++	**29**	**162**	25.6	55.3	**2.0**	**64.1**×
50	DDIM [46]	50	50	25.6	57.2	6.3	1.0×
	DDIM + ParaDiGMS [44]	21	132	25.6	56.9	3.3	1.9×
	DDIM + ParaSolver [35]	13	83	25.6	56.9	1.9	3.3×
	DDIM + ParaLin [36]	13	73	25.6	57.1	1.6	3.9×
	DDIM + ParaLin++	**10**	**69**	25.5	57.0	**1.3**	**4.8**×
25	DDIM [46]	25	25	25.4	62.9	3.4	1.0×
	DDIM + ParaDiGMS [44]	18	53	25.4	62.8	3.2	1.2×
	DDIM + ParaSolver [35]	11	49	25.4	61.9	1.2	2.8×
	DDIM + ParaLin [36]	11	42	25.4	61.8	1.1	3.1×
	DDIM + ParaLin++	**9**	**40**	25.4	61.9	**0.9**	**3.7**×
50	DPMSolver [31]	50	50	25.6	57.2	6.3	1.0×
	DPMSolver + ParaDiGMS [44]	25	132	25.6	57.2	3.3	1.8×
	DPMSolver + ParaSolver [35]	15	96	25.6	57.1	2.2	3.1×
	DPMSolver + ParaLin [36]	14	93	25.6	57.1	1.7	3.9×
	DPMSolver + ParaLin++	**10**	**77**	25.6	57.1	**1.2**	**5.2**×
25	DPMSolver [31]	25	25	25.4	62.3	3.2	1.0×
	DPMSolver + ParaDiGMS [44]	15	81	25.4	62.3	2.2	1.5×
	DPMSolver + ParaSolver [35]	11	58	25.4	62.2	1.5	2.1×
	DPMSolver + ParaLin [36]	10	55	25.4	62.1	1.2	2.7×
	DPMSolver + ParaLin++	**8**	**46**	25.4	62.1	**0.9**	**3.6**×

TABLE IV
QUANTITATIVE COMPARISONS OF DIFFERENT METHODS ON LSUN CHURCH OVER 5000 SAMPLES. DPMSOLVER IS NOT INTEGRATED WITH THE LSUN MODEL IN THE DIFFUSERS LIBRARY; THUS WE EXCLUDE IT.

Steps	Method	LSUN Church				
		Iters ↓	NFE ↓	FID↓	Time (s)↓	Speedup↑
1000	DDPM [20]	1000	1000	12.7	50.0	1.0×
	DDPM + ParaDiGMS [44]	65	2082	12.5	10.8	4.6×
	DDPM + ParaSolver [35]	42	1079	12.7	4.1	12.1×
	DDPM + ParaLin [36]	42	196	12.7	1.0	50.0×
	DDPM + ParaLin++	17	136	12.6	**0.7**	**71.4**×
50	DDIM [46]	50	50	15.5	1.8	1.0×
	DDIM + ParaDiGMS [44]	23	202	15.8	1.5	1.2×
	DDIM + ParaSolver [35]	14	73	15.7	0.8	2.2×
	DDIM + ParaLin [36]	14	67	15.7	0.6	3.0×
	DDIM + ParaLin++	9	64	15.7	**0.4**	**4.5**×
25	DDIM [46]	25	25	15.6	1.1	1.0×
	DDIM + ParaDiGMS [44]	15	96	15.7	0.8	1.4×
	DDIM + ParaSolver [35]	10	44	15.7	0.5	2.2×
	DDIM + ParaLin [36]	10	39	15.7	0.4	2.8×
	DDIM + ParaLin++	8	33	15.7	**0.3**	**3.6**×

TABLE V
LESS-STEP SAMPLING ON THE CIFAR-10 MODEL WITH 50K SAMPLES.

Method	Iters	FID↓	Speedup↑
DPM-Solver-v3 [61]	25	2.64	1.00×
DPM-Solver-v3 [61]	10	3.40	2.43×
UniPC [59]	10	3.40	2.41×
PD [43]	10	3.10	2.48×
CD [48]	10	2.90	2.46×
ParaLin++ + DPM-Solver-v3	10	**2.64**	2.41×

parallel variants when combined with ParaSolver, ParaDiGMS, ParaLin, and our ParaLin++, respectively. ParaLin++ demonstrates significant acceleration for DDPM, DDIM, and DPM-Solver while maintaining comparable FID and CLIP scores. Notably, it achieves a remarkable wall-clock time speedup of up to 64.1×, setting a new benchmark for parallel diffusion methods. Although parallelization inherently trades computational efficiency for speed, leading to increased NFEs relative to sequential approaches, our method capitalizes on partial linearity to substantially reduce NFEs, thereby unleashing the parallel efficiency.

Pixel-space Speedup Comparison. Similarly, Table IV shows that ParaLin++ consistently surpasses existing methods, delivering wall-clock speedups of 3.6×–71.4× on the LSUN Church model across the tested step configurations. At 1000 steps, ParaLin++ achieves a remarkable 71.4× speedup while preserving a nearly identical FID score (12.6 versus 12.7). For the 50-step and 25-step DDIM settings, it achieves 4.5× and 3.6× speedups, respectively. Notably, ParaLin++'s parallelism reduces iterations by 3.1×–58.8× relative to the sequential baselines and, among the parallel variants, keeps NFE low; in the 1000-step DDPM case, it uses 17 iterations and 136 NFEs versus the baseline's 1000. This acceleration is critical for real-time applications, such as interactive design tools, where latency directly impacts user experience.

Less-Step Sampling. Table V evaluates 10-step sampling on CIFAR-10 with 50,000 samples. ParaLin++ combined with DPM-Solver-v3 [61] reaches an FID of 2.64 and a 2.41× speedup, matching the 25-step DPM-Solver-v3 quality while improving over the compared 10-step baselines.

Industrial-Grade Benchmark Evaluation. The preceding FLUX FID comparison shows that ParaLin++ can accelerate a flow-matching model, but production deployment requires a stricter quality audit. Table VI therefore evaluates the same FLUX solvers on GenEval, DPG-Bench, and LongTextBench. UniPC and DPMSolver++ with 16 discrete steps preserve the Euler FID but lose 21%–24% relative GenEval score, 6.5–8.2 DPG-Bench points, and 67%–70% of the LongTextBench score. ParaLin++ instead keeps all 30 discrete diffusion steps and parallelizes the iterative solve: it closely preserves the Euler-level industrial-quality profile (0.64 versus 0.66 GenEval, 84.24 versus 83.97 DPG-Bench, and 0.26 versus 0.27 LongTextBench) and achieves a 1.68× GenEval-forward speedup. Hence, the acceleration is obtained by removing latency in the solver iteration rather than by shortening the diffusion discretization on which production-grade prompt and text fidelity depends.

This evaluation protocol also reflects the objectives of current industrial generators. Ming-Flash-Omni 2.0 [1], the sparse unified successor to Ming-Omni [2], reports 0.94 GenEval and 86.98 DPG-Bench, and its post-training explicitly optimizes realism, instruction adherence, and aesthetic quality together with task-specific rewards. These are precisely the production-level axes that FID does not reliably capture. Accordingly, the FLUX study and Ming-Flash-Omni 2.0 reference establish a more deployment-relevant test bed for judging whether a fast diffusion sampler merely matches a distributional statistic or actually preserves industrial generation quality. In addition to the motivating 16-step comparison, Table VII records the complete 16-, 20-, 24-, 26-, and 30-step industrial-benchmark matrices for FLUX and Ming-Flash-Omni 2.0. The table separates the number of discrete diffusion steps from the number of solver iterations: sequential samplers use one iteration per step, whereas ParaLin++ keeps 30 discrete steps and reports

TABLE VI
INDUSTRIAL MOTIVATION ON FLUX. FEW-STEP SEQUENTIAL SOLVERS PRESERVE FID BUT SHARPLY DEGRADE SEMANTIC COMPOSITION, COMPLEX PROMPT FOLLOWING, AND LONG-TEXT RENDERING. PARALIN++ KEEPS THE SAME 30 DISCRETE DIFFUSION STEPS AND PARALLELIZES THE SOLVER ITERATIONS.

Method	Sampling regime	Discrete steps	FID↓	GenEval↑	DPG-Bench↑	LongTextBench↑	Speedup↑
Euler [3]	Sequential reference	30	59.96	**0.66**	83.97	**0.27**	1.00×
UniPC [59]	Few-step	16	59.99	0.52	77.46	0.09	1.81×
DPMSolver++ [32]	Few-step	16	59.97	0.50	75.75	0.08	1.85×
ParaLin++	Step-parity parallel	30	**59.94**	0.64	**84.24**	0.26	1.68×

TABLE VII
DETAILED INDUSTRIAL-BENCHMARK RESULTS ON FLUX AND MING-FLASH-OMNI 2.0. WITHIN EACH MODEL BLOCK, SPEEDUP IS MEASURED AGAINST THAT MODEL'S 30-STEP EULER BASELINE ON THE SAME GENEVAL GENERATION PASS, EXCLUDING MODEL LOADING AND SCORING. FOR SEQUENTIAL SAMPLERS, "ITERS" EQUALS THE NUMBER OF DISCRETE STEPS; FOR PARALIN++, IT DENOTES THE NUMBER OF PARALLEL SOLVER ITERATIONS.

FLUX.1-dev [3]

Method	Sampling regime	Steps	Iters	GenEval↑	DPG-Bench↑	LongTextBench↑	Speedup↑
Euler [3]	Sequential reference	30	30	0.66	83.97	0.27	1.00×
UniPC [59]	Few-step	16	16	0.52	77.46	0.09	1.81×
		20	20	0.55	79.67	0.13	1.49×
		24	24	0.57	80.98	0.15	1.26×
		26	26	0.58	81.35	0.16	1.16×
		30	30	0.60	82.14	0.19	1.01×
DPMSolver++ [32]	Few-step	16	16	0.50	75.75	0.08	1.85×
		20	20	0.54	78.41	0.11	1.50×
		24	24	0.58	80.06	0.14	1.27×
		26	26	0.58	80.56	0.15	1.17×
		30	30	0.62	81.43	0.17	1.02×
ParaLin++	Step-parity parallel	30	17	0.64	84.24	0.26	1.68×

Ming-Flash-Omni 2.0 [1]

Method	Sampling regime	Steps	Iters	GenEval↑	DPG-Bench↑	LongTextBench↑	Speedup↑
Euler	Sequential reference	30	30	0.88	86.86	0.95	1.00×
UniPC [59]	Few-step	16	16	0.69	84.38	0.82	1.80×
		20	20	0.75	85.29	0.91	1.47×
		24	24	0.74	85.55	0.94	1.20×
		26	26	0.81	85.07	0.94	1.13×
		30	30	0.83	85.77	0.95	0.98×
DPMSolver++ [32]	Few-step	16	16	0.73	84.20	0.80	1.81×
		20	20	0.70	85.63	0.87	1.48×
		24	24	0.70	85.91	0.92	1.23×
		26	26	0.86	85.93	0.93	1.13×
		30	30	0.85	86.62	0.95	0.99×
ParaLin++	Step-parity parallel	30	24.69	0.87	86.99	0.95	1.21×

its average number of parallel solver iterations separately. For Ming-Flash-Omni 2.0, ParaLin++ reaches a 1.21× GenEval generation-pass speedup over the 30-step Euler reference while retaining the Euler-level profile: GenEval is 0.87 versus 0.88, DPG-Bench is 86.99 versus 86.86, and LongTextBench is 0.95 versus 0.95.

Table VIII extends the FLUX audit to different guidance strengths. Across the five CFG values, ParaLin++ obtains GenEval speedups of 1.55×–1.86× over the same-CFG 30-step Euler reference while preserving 97%–99% of the Euler GenEval score. Its GenEval scores are 0.59–0.66 and its LongTextBench scores are 0.15–0.26, close to Euler's respective ranges of 0.60–0.67 and 0.17–0.27. In contrast, the 20-step UniPC and DPMSolver++ configurations yield only 0.43–0.55 GenEval and 0.06–0.13 LongTextBench across the same CFG values. The DPG-Bench scores, now complete at all five CFG values, follow the same pattern: ParaLin++ tracks the Euler reference closely across CFG 0–14 (82.18–84.24 versus Euler's 82.25–83.97), whereas the 20-step UniPC and DPMSolver++ configurations drop to 68.85–79.67. The reference scores also show that guidance affects benchmarks differently: Euler's GenEval score peaks at CFG 10.5, whereas its LongTextBench score peaks at CFG 3.5. ParaLin++ remains close to the reference at every guidance setting. **(Takeaway)** Across CFG 0–14, the completed DPG-Bench matrix confirms that step-

TABLE VIII
FLUX.1-DEV CFG-SWEEP RESULTS. ALL BENCHMARK CELLS (GENEVAL, DPG-BENCH, AND LONGTEXTBENCH) ARE COMPLETE FOR ALL FIVE CFG VALUES; LTB ABBREVIATES LONGTEXTBENCH AND DPM++ ABBREVIATES DPMSOLVER++. "ITERS" EQUALS THE DISCRETE-STEP COUNT FOR SEQUENTIAL BASELINES AND DENOTES THE AVERAGE MEASURED PARALLEL SOLVER ITERATIONS FOR PARALIN++. SPEEDUP IS MEASURED AGAINST THE 30-STEP EULER REFERENCE AT THE SAME CFG ON THE GENEVAL GENERATION PASS, EXCLUDING MODEL LOADING AND SCORING.

CFG	Method	Iters	GenEval↑	DPG↑	LTB↑	Speedup↑
0	Euler	30	0.60	82.25	0.17	1.00×
	UniPC	20	0.46	78.97	0.08	1.49×
	DPM++	20	0.43	77.71	0.06	1.49×
	ParaLin++	14.79	0.59	82.18	0.15	1.86×
3.5	Euler	30	0.66	83.97	0.27	1.00×
	UniPC	20	0.55	79.67	0.13	1.49×
	DPM++	20	0.54	78.41	0.11	1.50×
	ParaLin++	16.92	0.64	84.24	0.26	1.68×
7	Euler	30	0.66	83.29	0.25	1.00×
	UniPC	20	0.49	72.60	0.07	1.49×
	DPM++	20	0.48	68.85	0.06	1.49×
	ParaLin++	17.85	0.64	83.22	0.23	1.55×
10.5	Euler	30	0.67	83.21	0.22	1.00×
	UniPC	20	0.52	77.89	0.07	1.48×
	DPM++	20	0.52	76.54	0.06	1.49×
	ParaLin++	17.03	0.66	83.23	0.21	1.62×
14	Euler	30	0.65	82.52	0.25	1.00×
	UniPC	20	0.54	75.88	0.08	1.49×
	DPM++	20	0.54	74.17	0.07	1.49×
	ParaLin++	17.13	0.64	82.75	0.23	1.61×

parity survives rising guidance while 20-step few-step solvers do not.

Figures 2 and 3 provide benchmark-specific visual inspections of the same quality axes. The FLUX examples use the exact 30-step reference/few-step settings compared above: Euler and ParaLin++ retain 30 discrete steps, while UniPC and DPMSolver++ use 16 steps. The examples show typical failures hidden by an FID-only criterion, such as missing or malformed objects, weaker dense-instruction compliance, and unreadable in-image text. The step-parity ParaLin++ outputs remain visually close to the 30-step Euler reference. For Ming-Flash-Omni 2.0, we show a uniform fully image-preserved qualitative-audit batch across all three benchmarks (guidance scale 7.0; Euler with 27 steps, UniPC and DPMSolver++ with 28 steps, and ParaLin++ with 30 steps); the CFG-5 quantitative matrix remains the primary setting reported in Table VII.

Figures 4 and 5 show fixed-prompt comparisons over all five CFG values. The same prompt and sample index are used in every column and row, so differences come from the method and CFG rather than from selecting a different example for each method. LongTextBench is represented by an English prompt because its reported metric averages the English and Chinese subsets. Figure 6 gives corresponding DPG-Bench samples at the two lowest guidance settings (CFG 0 and 3.5) for a representative prompt; the complete CFG-0–14 DPG-Bench matrix is reported in Table VIII.

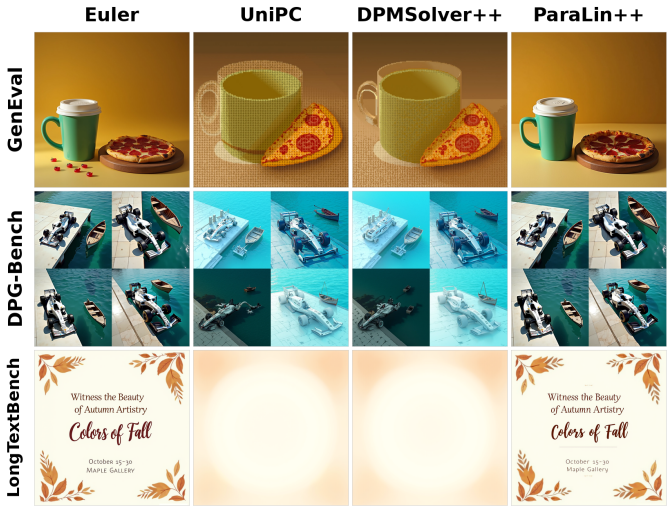

Fig. 2. Industrial-benchmark visual comparison on FLUX.1-dev. Each row is one representative benchmark prompt, and the columns show 30-step Euler, 16-step UniPC, 16-step DPMSolver++, and 30-step ParaLin++ (17 parallel solver iterations). Few-step examples retain plausible low-level appearance but can violate semantic composition, dense instructions, or in-image text requirements; ParaLin++ remains visually aligned with the 30-step Euler reference.

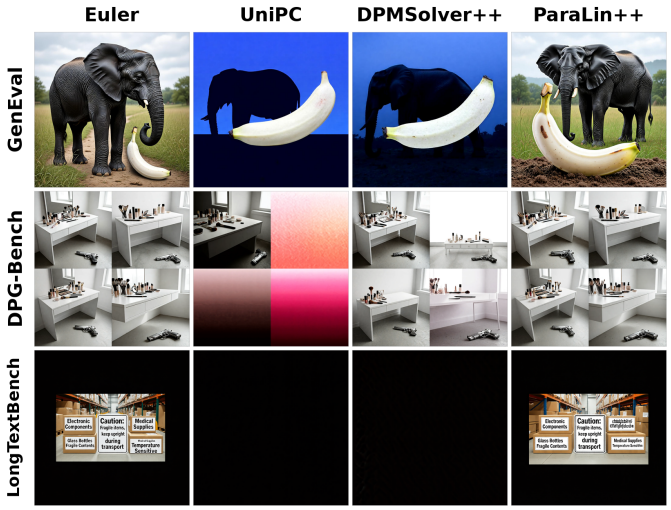

Fig. 3. Industrial-benchmark visual comparison on Ming-Flash-Omni 2.0. Each row is one representative benchmark prompt and the columns show Euler, UniPC, DPMSolver++, and ParaLin++. This uniform qualitative-audit batch uses guidance scale 7.0, Euler with 27 steps, UniPC and DPMSolver++ with 28 steps, and ParaLin++ with 30 steps; it complements, rather than replaces, the CFG-5 quantitative protocol in Table VII.

Single GPU for Acceleration Parallel sampling methods are often criticized for trading excessive computational cost for speed, which is impractical for users with limited resources. However, our experiments in Table IX demonstrate the real-world viability of our approach on a single GPU. Unlike competing parallel methods like ParaDiGMS, which often fail to provide any speedup or even decelerate the process in this setting, our ParaLin++ achieves consistent and significant wall-clock acceleration across all baseline samplers. For instance, it speeds up the 100-step DDIM by 4.2× and the 100-step DPMSolver by 3.7×, while remarkably accelerating the 1000-

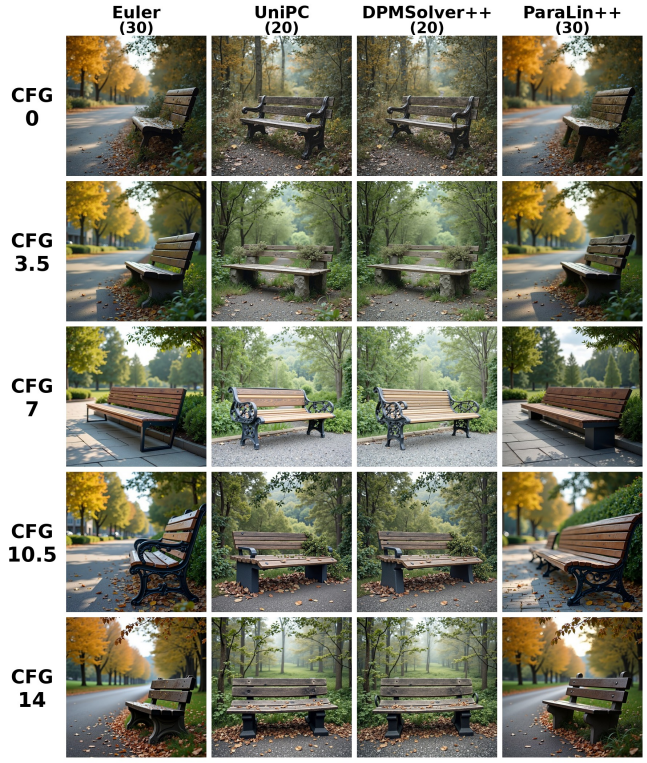

Fig. 4. GenEval CFG-sweep visual comparison on FLUX.1-dev. Rows are CFG 0, 3.5, 7, 10.5, and 14; columns are 30-step Euler, 20-step UniPC, 20-step DPMSolver++, and 30-step ParaLin++. All cells use prompt 00000 and sample 0000.

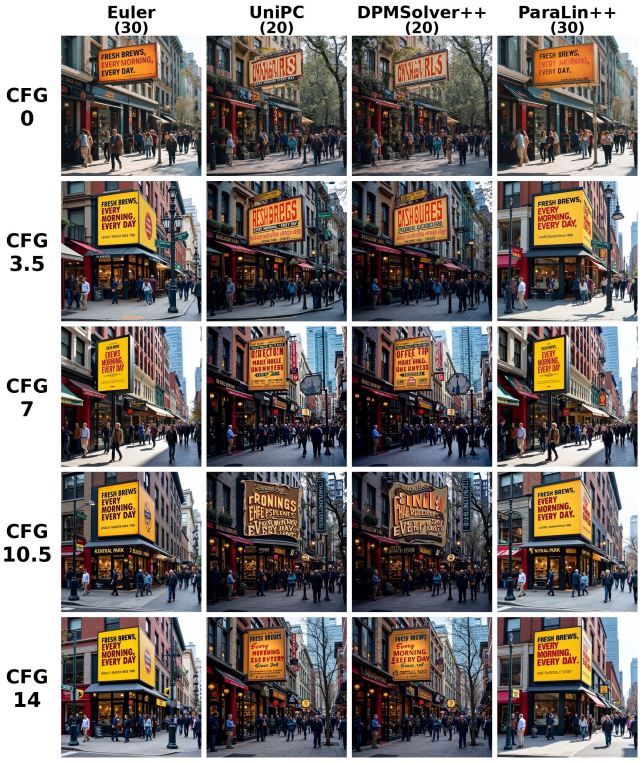

Fig. 5. LongTextBench CFG-sweep visual comparison on FLUX.1-dev, using the English prompt 0000 with variant 1. Rows are CFG 0, 3.5, 7, 10.5, and 14; columns are 30-step Euler, 20-step UniPC, 20-step DPMSolver++, and 30-step ParaLin++. The reported LongTextBench metric averages the English and Chinese scores.

step DDPM by $18.0\times$. This is accomplished while maintaining virtually identical generation quality (FID and CLIP scores). This validates ParaLin++ as a practical and potent accelerator for everyday users, effectively overcoming the typical computational barriers of parallel sampling even in resource-constrained, single-GPU environments.

Ablation Study. Table X isolates the value of the ParaLin++ components on a 100-step DDIM task. Full nonlinearity reduces iterations but multiplies NFEs and becomes slower than the sequential baseline. Partial nonlinearity removes 93.3% of these NFEs relative to the fully nonlinear variant and already restores wall-clock speedup. Adaptive stopping slightly reduces both iterations and NFEs, while SOR gives the strongest reduction, reaching 15 iterations, 117 NFEs, and a $6.5\times$ speedup. The trend confirms that the largest gain comes from ParaLin's partial-linear formulation, while ParaLin++ further improves convergence by lowering iterations with negligible extra computation.

GPU Scaling Analysis. Table XI examines GPU scaling performance. Our method achieves $71.4\times$ speedup, significantly outperforming existing approaches. Notably, our approach delivers outstanding performance even on a single GPU, achieving $41.6\times$ acceleration. This single-GPU efficiency offers significant practical advantages, enabling high-speed generation for resource-constrained scenarios. The efficiency stems from our linearized sampling process that solves only a partial nonlinear system.

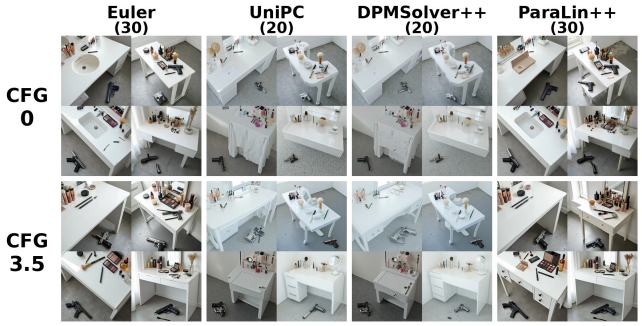

Fig. 6. DPG-Bench CFG visual comparison on FLUX.1-dev at the two lowest guidance settings. Rows show CFG 0 and 3.5; columns are 30-step Euler, 20-step UniPC, 20-step DPMSolver++, and 30-step ParaLin++. All cells use prompt 100. The complete CFG-0–14 DPG-Bench matrix is reported in Table VIII.

Generalization on VP, VE, and EDM Process. We investigated the impact of the VP, VE, and EDM diffusion processes by converting the sequential sampler in Alg. 1 of work [22] into our parallel version. Adopting the Tab. 1 configurations specified in work [22], we then analyzed the effect of the nonlinear degree ϱ on FID performance across these three processes using the provided ImageNet dataset. As shown in Tab. XII, our parallel sampling method demonstrates its applicability and effectiveness across all three diffusion processes. For each process, our parallel approach consistently reduces the number of sampling iterations while achieving comparable

TABLE IX
COMPARISONS ON STABLE DIFFUSION-V2 WHEN ONLY USING ONE GPU, WITH CLASSIFIER GUIDANCE W = 7.5.

Steps	Method	Stable Diffusion-v2					
		Iters ↓	NFE ↓	CLIP↑	FID↓	Time (s)↓	Speedup↑
1000	DDPM	1000	1000	25.6	55.9	128.0	1.0×
	DDPM + ParaDiGMS [44]	65	2024	25.6	55.6	153.6	0.8×
	DDPM + ParaSolver [35]	52	1021	25.6	55.3	84.1	1.5×
	DDPM + ParaLin [36]	41	221	25.6	55.7	23.1	5.7×
	DDPM + ParaLin++	**20**	**80**	25.6	55.7	**7.1**	**18.0**×
50	DDIM [46]	50	50	25.6	57.2	6.3	1.0×
	DDIM + ParaDiGMS [44]	34	67	25.6	56.9	7.8	0.8×
	DDIM + ParaSolver [35]	18	53	25.6	56.9	6.0	1.05×
	DDIM + ParaLin [36]	14	51	25.6	57.1	5.7	1.1×
	DDIM + ParaLin++	**8**	**32**	25.6	57.1	**3.2**	**2.0**×
100	DDIM [46]	100	100	25.6	56.3	10.1	1.0×
	DDIM + ParaDiGMS [44]	66	131	25.5	56.2	12.0	0.8×
	DDIM + ParaSolver [35]	35	102	25.5	56.3	8.5	1.2×
	DDIM + ParaLin [36]	18	70	25.6	56.3	6.1	1.7×
	DDIM + ParaLin++	**11**	**36**	25.5	56.4	**2.4**	**4.2**×
50	DPMSolver [31]	50	50	25.6	57.2	5.9	1.0×
	DPMSolver + ParaDiGMS [44]	27	78	25.6	57.2	6.1	0.9×
	DPMSolver + ParaSolver [35]	33	65	25.6	57.2	6.0	0.9×
	DPMSolver + ParaLin [36]	31	56	25.5	56.9	5.4	1.1×
	DPMSolver + ParaLin++	**11**	**28**	25.6	57.1	**2.3**	**2.5**×
100	DPMSolver [31]	100	100	25.5	56.7	9.9	1.0×
	DPMSolver + ParaDiGMS [44]	66	131	25.6	56.6	11.9	0.8×
	DPMSolver + ParaSolver [35]	64	127	25.5	56.6	10.7	0.9×
	DPMSolver + ParaLin [36]	64	121	25.6	56.7	10.0	1.0×
	DPMSolver + ParaLin++	**14**	**40**	25.6	56.5	**2.7**	**3.7**×

TABLE X
ABLATION STUDY ON STABLE DIFFUSION V2. THE TABLE ISOLATES NONLINEARITY, ADAPTIVE STOPPING, AND SOR IN THE 100-STEP DDIM TASK; SPEEDUP IS MEASURED RELATIVE TO THE SEQUENTIAL DDIM BASELINE.

Configuration	Solver components			100-step DDIM results			
	Nonlinearity	Stopping criterion	SOR	Iters↓	NFE↓	Time (s)↓	Speedup↑
Sequential DDIM	–	–	–	100	100	10.6	1.0×
Full nonlinearity	Full ($\varrho = 100$)	Fixed ($\beta = 1.0$)	No ($\omega = 1.0$)	72	2100	24.4	0.43×
Partial nonlinearity	Partial ($\varrho = 8$)	Fixed ($\beta = 1.0$)	No ($\omega = 1.0$)	18	140	2.1	5.0×
Partial + adaptive stopping	Partial ($\varrho = 8$)	Adaptive ($\beta = 0.999$)	No ($\omega = 1.0$)	17	139	2.0	5.3×
ParaLin++ (partial + adaptive + SOR)	Partial ($\varrho = 8$)	Adaptive ($\beta = 0.999$)	Yes ($\omega = 1.04$)	**15**	**117**	**1.6**	**6.5**×

or improved FID scores compared to the sequential baseline. This highlights the robustness and efficiency of our method, making it a viable accelerator for various diffusion model formulations. We notice that our method yields a better FID score for every schedule and a substantially larger gain in the VP process. This improvement likely stems from the inherently stiff nature of the VP-ODE, combined with the step-parallel updates of our parallel solver. The step-parallel updates sever the temporal dependence chain across steps, thus effectively curtail the accumulation of truncation error across steps [22]. As a result, our solver acts as an implicit numerical stabilizer for the stiff VP-ODE, which typically exhibits significant truncation error that degrades the performance of conventional VP samplers.

Nonlinear Degree ϱ Analysis: Table XIII conducted an ablation study by accelerating a 50-step DPMSolver on the Stable Diffusion v2 model using 8 GPUs. As shown in the table, the FID and CLIP scores remain remarkably stable across a wide range of ϱ values. This stability confirms our theoretical finding in Prop. 2 that ϱ does not compromise the fidelity of the generated samples. Furthermore, the results reveal a clear trade-off between ϱ and efficiency. While increasing ϱ initially yields significant speedups, the gains diminish and eventually reverse. This is because a larger ϱ requires more NFEs, as seen in the table. The increased NFE count results in a greater computational overhead for each GPU (proportional to NFEs / GPUs), which leads to a rapid increase in runtime after an optimal point (around $\varrho = 15$).

EMA Decay Rate β and Tolerance δ Analysis. As shown in Table XIV, the EMA decay rate (β) is critical for sample quality. A low value ($\beta = 0.9$) causes noticeable FID degradation, whereas higher values ($\beta \geqslant 0.99$) consistently achieve FIDs comparable to or better than the baseline. Conversely, for the practical high-decay choices ($\beta \geqslant 0.99$), the tolerance (δ) has only a small impact on the final FID score. This reveals a key practical trade-off: δ can be relaxed (set higher)

TABLE XI
THE EFFECT OF THE NUMBER OF GPUS USING LSUN CHURCH WHEN EXPEDITING 1000 STEPS' DDPM.

Method	1 GPU		2 GPUs		4 GPUs		8 GPUs	
	Time (s)↓	Speedup↑	Time (s)↓	Speedup↑	Time (s)↓	Speedup↑	Time (s)↓	Speedup↑
DDPM [20]	50.0	1.0×	50.0	1.0×	50.0	1.0×	50.0	1.0×
ParaDiGMS [44]	37.0	1.4×	23.3	2.1×	15.2	3.3×	10.8	4.6×
ParaSolver [35]	19.7	2.5×	13.0	3.8×	9.9	5.1×	4.1	12.1×
ParaLin [36]	4.5	11.1×	2.0	25.0×	1.6	31.3×	1.0	50.0×
ParaLin++	**1.2**	**41.6**×	**1.0**	**50.0**×	**0.9**	**55.5**×	**0.7**	**71.4**×

TABLE XII
THE IMPACT OF NONLINEAR DEGREE ϱ ON FID ACROSS EDM, VE, AND VP PROCESSES OVER IMAGENET DATASET. WE SET $\delta = 0.01, \beta = 9e-4$ FOR THE EDM PROCESS, $\delta = 0.01, \beta = 9e-7$ FOR VP PROCESSES, AND $\delta = 0.1, \beta = 9e-5$ FOR THE VE PROCESS.

Method	EDM Process		VE Process		VP Process	
	FID	Iters	FID	Iters	FID	Iters
Baseline	11.1649	25	423.0628	25	104.14673	25
$\varrho = 4$	10.8811	22	422.9794	14	35.8963	12
$\varrho = 6$	10.8718	21	422.9293	14	41.1196	10
$\varrho = 8$	10.8621	20	422.8247	13	58.5588	8
$\varrho = 10$	10.8626	15	422.7672	10	58.5215	7
$\varrho = 12$	10.8618	12	422.7651	9	58.8340	7

TABLE XIII
IMPACT OF VARYING ϱ ON GENERATION QUALITY (FID, CLIP) AND PERFORMANCE (NFES, SPEEDUP).

Method	FID	CLIP	Iters	NFEs	Speedup
Baseline	57.23	25.64	50	50	1×
$\varrho = 5$	57.18	25.55	18	82	2.9×
$\varrho = 10$	57.25	25.59	12	101	3.0×
$\varrho = 15$	57.14	25.61	11	123	3.1×
$\varrho = 20$	57.22	25.60	12	160	2.6×
$\varrho = 25$	57.14	25.53	12	202	2.1×
$\varrho = 30$	57.06	25.58	12	223	2.1×
$\varrho = 35$	57.18	25.62	12	245	1.8×
$\varrho = 40$	57.15	25.61	12	264	1.8×
$\varrho = 45$	57.16	25.59	12	275	1.6×
$\varrho = 50$	57.14	25.57	12	284	1.6×

to significantly reduce the number of iterations and thus accelerate computation, for example, from 21 to 11 iterations when $\beta = 1.0$, without a meaningful sacrifice in generation quality.

TABLE XIV
THE EFFECT OF THE TOLERANCE δ AND EMA DECAY RATE β ON FID USING STABLE DIFFUSION-V2 OVER 5000 RANDOM SAMPLES. OUR PARALIN++ ($\varrho = 8, p = 40$) IS EMPLOYED FOR DDIM.

Method	$\delta = 0.01$		$\delta = 0.05$		$\delta = 0.1$	
	FID	Iters	FID	Iters	FID	Iters
DDIM	57.2	50	57.2	50	57.2	50
$\beta = 0.9$	59.7	9	58.6	9	57.8	9
$\beta = 0.99$	57.8	13	57.1	13	56.9	11
$\beta = 0.999$	57.0	16	56.9	13	56.8	11
$\beta = 1.0$	57.0	21	57.1	13	57.2	11

Relaxation Factor ω Analysis. Table XV demonstrates the trade-off between acceleration and stability controlled by the relaxation factor ω. Increasing ω from 1.0 to 1.08 effectively reduces the number of iterations and, consequently, boosts the speedup from $3.81\times$ to a peak of $4.94\times$. However, this gain is bounded by stability. Sample quality, as measured by FID and CLIP, remains consistent with the baseline up to $\omega = 1.06$. At $\omega = 1.08$, a slight degradation in quality appears (FID 58.13), and by $\omega = 1.12$, the method becomes completely unstable, leading to a catastrophic collapse in generation quality (FID 289.54). This suggests an optimal value around $\omega = 1.06$, which achieves a substantial $4.76\times$ speedup while fully preserving the original sample fidelity.

TABLE XV
IMPACT OF VARYING ω ON GENERATION QUALITY AND PERFORMANCE. WE SET $\varrho = 8, p = 40, \beta = 0.999$.

Method	FID	CLIP	Iters	NFEs	Speedup
DDIM	57.23	25.64	50	50	1×
$\omega = 1$	57.20	25.59	11	83	3.81×
$\omega = 1.02$	57.38	25.57	11	79	3.89×
$\omega = 1.04$	57.38	25.57	10	70	4.34×
$\omega = 1.06$	57.24	25.56	9	67	4.76×
$\omega = 1.08$	58.13	25.12	9	67	4.94×
$\omega = 1.12$	289.54	15.03	8	64	4.3×

Figure 7 provides qualitative samples for the corresponding ϱ, β, and ω ablations; the last image in each row is the DDIM baseline.

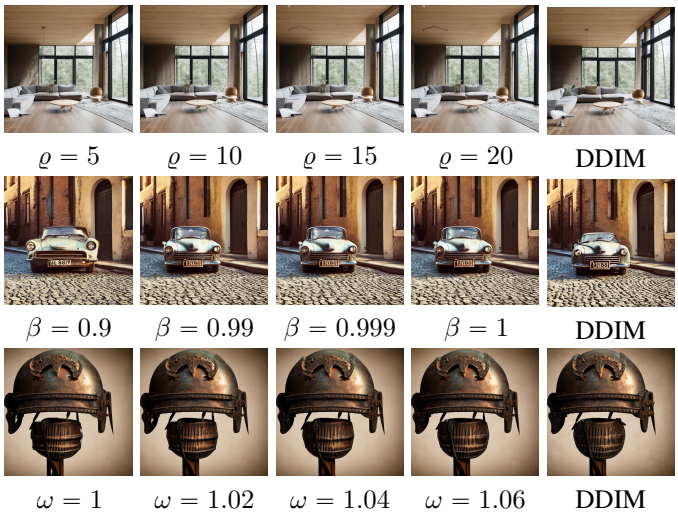

Fig. 7. Ablation study on parameters. The first row shows the results for different values of ϱ. The second row shows the results for different values of β. The third row shows the results for different values of ω. The last image in each row represents the baseline comparison.

VI. Conclusion

We systematized training-free diffusion acceleration by the quality metrics that certify it. The academic literature certifies speedups almost exclusively with FID/CLIP-class distributional proxies, yet our industrial audit shows that an FID-matched few-step sampler is not necessarily production-ready: 16-step UniPC/DPMSolver++ keep FID within 0.03 of the 30-step Euler reference while losing up to 24% GenEval, 8.2 DPG-Bench points, and 70% LongTextBench. We therefore advocate reporting an industrial-quality profile alongside FID, separating discrete steps from solver iterations, and preferring the step-parity route when prompt or text fidelity is part of the service-level objective. ParaLin++, our strongest realization of this route, parallelizes the full 30-step discretization and is backed by an SOR-stability range and an accumulated error bound; it preserves the Euler-level industrial profile at $1.68\times$ on FLUX.1-dev and $1.21\times$ on Ming-Flash-Omni 2.0, and reaches $3.6\times$–$5.2\times$ (up to $71.4\times$ at 1000 steps) on Stable Diffusion-v2 and LSUN Church. Remaining limitations include inter-GPU communication overhead at large ϱ and device counts, and the iterative nature of parallel solving, which distillation toward few-step generation could address.

References

[1] I. AI, "Ming-flash-omni: A sparse, unified architecture for multimodal perception and generation," *arXiv preprint arXiv:2510.24821*, 2025.

[2] ——, "Ming-omni: A unified multimodal model for perception and generation," 2025. [Online]. Available: https://arxiv.org/abs/2506.09344

[3] Black Forest Labs, "Introducing flux.1," https://www.blackforestlabs.ai/announcements/introducing-flux, August 2024, accessed: 2025-09-07.

[4] A. Blattmann, T. Dockhorn, S. Kulal, D. Mendelevitch, M. Kilian, D. Lorenz, Y. Levi, Z. English, V. Voleti, A. Letts *et al.*, "Stable video diffusion: Scaling latent video diffusion models to large datasets," *arXiv preprint arXiv:2311.15127*, 2023.

[5] W. E. Boyce, R. C. DiPrima, and D. B. Meade, *Elementary differential equations and boundary value problems*. John Wiley & Sons, 2021.

[6] R. L. Burden and J. D. Faires, "Fixed-point iteration," *Numerical Analysis (3rd ed.). PWS Publishers*, p. 64, 1985.

[7] C.-T. Chen, *Linear system theory and design*. Saunders college publishing, 1984.

[8] H. Chen, Y. Ren, L. Ying, and G. M. Rotskoff, "Accelerating diffusion models with parallel sampling: Inference at sub-linear time complexity," *arXiv preprint arXiv:2405.15986*, 2024.

[9] S. Chen, S. Chewi, J. Li, Y. Li, A. Salim, and A. R. Zhang, "Sampling is as easy as learning the score: theory for diffusion models with minimal data assumptions," in *International Conference on Learning Representations*, 2023.

[10] H. Chung, J. Kim, S. Kim, and J. C. Ye, "Parallel diffusion models of operator and image for blind inverse problems," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 6059–6069.

[11] N. Elata, B. Kawar, T. Michaeli, and M. Elad, "Nested diffusion processes for anytime image generation," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 5018–5027.

[12] Z. Geng, A. Pokle, and J. Z. Kolter, "One-step diffusion distillation via deep equilibrium models," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[13] Z. Geng, Y. Wang, Y. Ma, C. Li, Y. Rao, S. Gu, Z. Zhong, Q. Lu, H. Hu, X. Zhang, Linus, D. Wang, and J. Jiang, "X-omni: Reinforcement learning makes discrete autoregressive image generative models great again," *arXiv preprint arXiv:2507.22058*, 2025.

[14] D. Ghosh, H. Hajishirzi, and L. Schmidt, "Geneval: An object-focused framework for evaluating text-to-image alignment," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[15] M. Gong, S. Xie, W. Wei, M. Grundmann, K. Batmanghelich, T. Hou *et al.*, "Semi-implicit denoising diffusion models (siddms)," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[16] M. Gonzalez, N. Fernandez Pinto, T. Tran, H. Hajri, N. Masmoudi *et al.*, "Seeds: Exponential sde solvers for fast high-quality sampling from diffusion models," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[17] S. Gupta, L. Cai, and S. Chen, "Faster diffusion-based sampling with randomized midpoints: Sequential and parallel," *arXiv preprint arXiv:2406.00924*, 2024.

[18] J. Hessel, A. Holtzman, M. Forbes, R. L. Bras, and Y. Choi, "Clipscore: A reference-free evaluation metric for image captioning," *arXiv preprint arXiv:2104.08718*, 2021.

[19] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "Gans trained by a two time-scale update rule converge to a local nash equilibrium," *Advances in neural information processing systems*, vol. 30, 2017.

[20] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.

[21] X. Hu, R. Wang, Y. Fang, B. Fu, P. Cheng, and G. Yu, "Ella: Equip diffusion models with llm for enhanced semantic alignment," *arXiv preprint arXiv:2403.05135*, 2024.

[22] T. Karras, M. Aittala, T. Aila, and S. Laine, "Elucidating the design space of diffusion-based generative models," in *Proc. NeurIPS*, 2022.

[23] C. T. Kelley, *Solving nonlinear equations with Newton's method*. SIAM, 2003.

[24] M. Li, T. Cai, J. Cao, Q. Zhang, H. Cai, J. Bai, Y. Jia, K. Li, and S. Han, "Distrifusion: Distributed parallel inference for high-resolution diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 7183–7193.

[25] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *The Eleventh International Conference on Learning Representations*, 2023.

[26] K. Liu, Z. Zhu, C. Li, H. Liu, H. Zeng, and J. Hou, "Prefpaint: Aligning image inpainting diffusion model with human preference," *Advances in Neural Information Processing Systems*, vol. 37, pp. 30 554–30 589, 2025.

[27] X. Liu, C. Gong *et al.*, "Flow straight and fast: Learning to generate and transfer data with rectified flow," in *The Eleventh International Conference on Learning Representations*, 2023.

[28] X. Liu, X. Zhang, J. Ma, J. Peng *et al.*, "Instaflow: One step is enough for high-quality diffusion-based text-to-image generation," in *The Twelfth International Conference on Learning Representations*, 2023.

[29] ——, "Instaflow: One step is enough for high-quality diffusion-based text-to-image generation," in *The Twelfth International Conference on Learning Representations*, 2023.

[30] X. Liu, Z. Li, and Q. Gu, "Cachequant: Comprehensively accelerated diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.

[31] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu, "Dpm-solver: A fast ode solver for diffusion probabilistic model sampling in around 10 steps," *Advances in Neural Information Processing Systems*, vol. 35, pp. 5775–5787, 2022.

[32] ——, "Dpm-solver++: Fast solver for guided sampling of diffusion probabilistic models," *arXiv preprint arXiv:2211.01095*, 2022.

[33] J. Lu, S. Hu, W. Wan, M. Li, L. Y. Zhang, L. Xue, and H. Jin, "Depriving the survival space of adversaries against poisoned gradients in federated learning," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 5405–5418, 2024.

[34] J. Lu, L. Xue, W. Wan, M. Li, L. Y. Zhang, and S. Hu, "Preserving privacy of input features across all stages of collaborative learning," in *ISPA/BDCloud/SocialCom/SustainCom*, 2023.

[35] J. Lu, Z. Zhu, and J. Hou, "Parasolver: A hierarchical parallel integral solver for diffusion models," in *The Thirteenth International Conference on Learning Representations*, 2025.

[36] J. Lu, Z. Zhu, H. Liu, and J. Hou, "Paralin: Accelerating parallel diffusion integrator via intrinsic partially linear structure," in *Advances in Neural Information Processing Systems*, 2026.

[37] S. Luo, Y. Tan, L. Huang, J. Li, and H. Zhao, "Latent consistency models: Synthesizing high-resolution images with few-step inference," *arXiv preprint arXiv:2310.04378*, 2023.

[38] X. Ma, G. Fang, M. Bi Mi, and X. Wang, "Learning-to-cache: Accelerating diffusion transformer via layer caching," *Advances in Neural Information Processing Systems*, vol. 37, pp. 133 282–133 304, 2024.

[39] X. Ma, G. Fang, and X. Wang, "Deepcache: Accelerating diffusion models for free," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 15 762–15 772.
[40] L. X. Nguyen, P. S. Aung, H. Q. Le, S.-B. Park, and C. S. Hong, "A new chapter for medical image generation: the stable diffusion method," in *2023 International Conference on Information Networking (ICOIN)*. IEEE, 2023, pp. 483–486.
[41] Z. Pan, B. Zhuang, D.-A. Huang, W. Nie, Z. Yu, C. Xiao, J. Cai, and A. Anandkumar, "T-stitch: Accelerating sampling in pre-trained diffusion models with trajectory stitching," *arXiv preprint arXiv:2402.14167*, 2024.
[42] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 10 684–10 695.
[43] T. Salimans and J. Ho, "Progressive distillation for fast sampling of diffusion models," in *International Conference on Learning Representations*, 2022.
[44] A. Shih, S. Belkhale, S. Ermon, D. Sadigh, and N. Anari, "Parallel sampling of diffusion models," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
[45] J. So, J. Shin, C. Jang, and E. Park, "Pcm: Picard consistency model for fast parallel sampling of diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.
[46] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *International Conference on Learning Representations*, 2021.
[47] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever, "Consistency models," *International Conference on Machine Learning*, 2023.
[48] ——, "Consistency models," in *International Conference on Machine Learning*. PMLR, 2023, pp. 32 211–32 252.
[49] Y. Song, C. Meng, R. Liao, and S. Ermon, "Accelerating feedforward computation via parallel nonlinear equation solving," in *International Conference on Machine Learning*. PMLR, 2021, pp. 9791–9800.
[50] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," in *International Conference on Learning Representations*, 2021.
[51] Z. Tang, J. Tang, H. Luo, F. Wang, and T.-H. Chang, "Accelerating parallel sampling of diffusion models," in *Forty-first International Conference on Machine Learning*, 2024.
[52] VincentGOURBIN, "Fluxprompting," https://huggingface.co/datasets/VincentGOURBIN/FluxPrompting, 2024.
[53] G. Wang, Y. Cai, L. Li, W. Peng, and S. Su, "Pfdiff: Training-free acceleration of diffusion models through the gradient guidance of past and future," in *The Thirteenth International Conference on Learning Representations*, 2025.
[54] J. Wang, J. Fang, A. Li, and P. Yang, "Pipefusion: Displaced patch pipeline parallelism for inference of diffusion transformer models," *arXiv preprint arXiv:2405.14430*, 2024.
[55] J. Wu, R. Fu, H. Fang, Y. Zhang, Y. Yang, H. Xiong, H. Liu, and Y. Xu, "Medsegdiff: Medical image segmentation with diffusion probabilistic model," in *Medical Imaging with Deep Learning*. PMLR, 2024, pp. 1623–1639.
[56] R. Yang, Y. Yang, F. Zhou, and Q. Sun, "Directional diffusion models for graph representation learning," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
[57] M. You, Z. Zhu, H. Liu, and J. Hou, "Nvs-solver: Video diffusion model as zero-shot novel view synthesizer," *International Conference on Learning Representations*, 2025.
[58] J. Yu and H. Barad, "Step saver: Predicting minimum denoising steps for diffusion model image generation," *arXiv preprint arXiv:2408.02054*, 2024.
[59] W. Zhao, L. Bai, Y. Rao, J. Zhou, and J. Lu, "Unipc: A unified predictor-corrector framework for fast sampling of diffusion models," *Advances in Neural Information Processing Systems*, vol. 36, pp. 49 842–49 869, 2023.
[60] H. Zheng, W. Nie, A. Vahdat, K. Azizzadenesheli, and A. Anandkumar, "Fast sampling of diffusion models via operator learning," in *International conference on machine learning*. PMLR, 2023, pp. 42 390–42 402.
[61] K. Zheng, C. Lu, J. Chen, and J. Zhu, "Dpm-solver-v3: Improved diffusion ode solver with empirical model statistics," *Advances in Neural Information Processing Systems*, vol. 36, pp. 55 502–55 542, 2023.
[62] H. Zhou and M. Sugiyama, "Parallel simulation for sampling under isoperimetry and score-based diffusion models," *arXiv preprint arXiv:2412.07435*, 2024.

Jianrong Lu received the M.S degree in cyberspace security from Huazhong University of Science and Technology, Wuhan, China, in 2023. He is currently a Research Assistant at City University of Hong Kong. His research interests include diffusion model and stochastic optimization.

Zhiyu Zhu is an Assistant Professor with City University of Hong Kong (Dongguan). He received the B.E. and M.E. degrees in Mechatronic Engineering, both from Harbin Institute of Technology, in 2017 and 2019, respectively, and a Ph.D. degree in Computer Science from City University of Hong Kong in 2023. His research interests include generative models and computer vision.

Hui Liu is currently an Assistant Professor with the School of Computing and Information Sciences at Saint Francis University, Hong Kong SAR, China. She received the B.Sc. degree in Communication Engineering from Central South University, Changsha, China, the M.Eng. degree in Computer Science from Nanyang Technological University, Singapore, and the Ph.D. degree from the Department of Computer Science, City University of Hong Kong, Hong Kong. Her research interests include image processing and machine learning.

Junhui Hou (Senior Member, IEEE) is a Professor with the Department of Computer Science, City University of Hong Kong (CityUHK). His research interests include multidimensional visual computing and machine learning.

He received the Early Career Award and the Research Fellow Award from the Hong Kong Research Grants Council, the Excellent Young Scientists Fund from the National Natural Science Foundation of China, the IEEE TIP Best Paper Award, and the CityUHK Presidential Research Excellence Award for Junior Faculty. He is serving as a Senior Area Editor for IEEE TIP and an Associate Editor for IEEE TVCG and TMM. He served as an Associate Editor for IEEE TIP and TCSVT.

APPENDIX A
PROOF OF PROPOSITION 1

A. Derivation of the Error Bound

Before proceeding with the proof, we first state the following practical assumption regarding the denoising model.

Assumption 1. *The denoising network $\mathbf{S}_{\theta*}(\mathbf{X}_t, t)$ is a well-trained noise predictor. Specifically, for any noisy sample $\mathbf{X}_t = \alpha_t \mathbf{X}_T + \sigma_t \epsilon_t$ (where $\mathbf{X}_T$ is the clean sample and $\epsilon_t \sim \mathcal{N}(0, \mathbf{I})$ is the specific noise instance), there exists a upper bound $\zeta > 0$ such that the model's prediction error is bounded:*

$$\|\mathbf{S}_{\theta*}(\mathbf{X}_t, t) - \epsilon_t\| < \zeta$$

We recall the forward process $p(\mathbf{X}_t|\mathbf{X}_T) = \mathcal{N}(\alpha_t \mathbf{X}_T, \sigma_t^2 \mathbf{I})$, where α_t and σ_t are the noise schedule coefficients. We define the log-SNR as $\lambda_t = \log(\alpha_t/\sigma_t)$. It follows that $e^{-\lambda_t} = \sigma_t/\alpha_t$ and λ_t is a strictly decreasing function of t.

Following Proposition 3.1 in DPM-Solver [31], the exact solution of the corresponding SDE from time s to t is given by:

$$\begin{aligned}\Phi(t, s, \mathbf{X}_s) = \frac{\alpha_t}{\alpha_s}\mathbf{X}_s + \alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda} \sigma_\lambda \nabla_{\mathbf{X}_\lambda} \log p(\mathbf{X}_\lambda) d\lambda \\ + \int_{\lambda_s}^{\lambda_t} g(\lambda) d\mathbf{W},\end{aligned} \tag{29}$$

where $p(\mathbf{X}_\lambda)$ is the marginal distribution. In a practical solver, the true score $\nabla_{\mathbf{X}_\lambda} \log p(\mathbf{X}_\lambda) = -\mathbb{E}[\epsilon|\mathbf{X}_\lambda]/\sigma_\lambda$ is unavailable and is replaced by the model's prediction. Specifically, the solver approximates the score using the noise prediction model $\mathbf{S}_{\theta*}$:

$$\hat{\nabla}_{\mathbf{X}_\lambda} \log p(\mathbf{X}_\lambda) = -\frac{\mathbf{S}_{\theta*}(\mathbf{X}_\lambda, \lambda)}{\sigma_\lambda}$$

By substituting this approximation into Eq. (29), we define the **practical sequential solver** $\hat{\Phi}$ as:

$$\begin{aligned}\hat{\Phi}(t, s, \mathbf{X}_s) &= \frac{\alpha_t}{\alpha_s}\mathbf{X}_s + \alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda} \sigma_\lambda \left(-\frac{\mathbf{S}_{\theta*}(\mathbf{X}_\lambda, \lambda)}{\sigma_\lambda}\right) d\lambda \\ &\quad + \int_{\lambda_s}^{\lambda_t} g(\lambda) d\mathbf{W} \\ &= \frac{\alpha_t}{\alpha_s}\mathbf{X}_s - \alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda} \mathbf{S}_{\theta*}(\mathbf{X}_\lambda, \lambda) d\lambda + \int_{\lambda_s}^{\lambda_t} g(\lambda) d\mathbf{W}.\end{aligned} \tag{30}$$

Our goal is to show that this practical solver $\hat{\Phi}$ can be precisely approximated by a linear solver. We use Assumption 1 to decompose the model's prediction $\mathbf{S}_{\theta*}$ into the ground-truth noise ϵ_λ and a bounded error term $\delta(\mathbf{X}_\lambda, \lambda)$:

$$\mathbf{S}_{\theta*}(\mathbf{X}_\lambda, \lambda) = \epsilon_\lambda + \delta(\mathbf{X}_\lambda, \lambda),$$

where $\|\delta(\mathbf{X}_\lambda, \lambda)\| < \zeta$.

Substituting this decomposition back into our practical solver Eq. (30):

$$\begin{aligned}\hat{\Phi}(t, s, \mathbf{X}_s) &= \frac{\alpha_t}{\alpha_s}\mathbf{X}_s - \alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda} \left(\epsilon_\lambda + \delta(\mathbf{X}_\lambda, \lambda)\right) d\lambda \\ &\quad + \int_{\lambda_s}^{\lambda_t} g(\lambda) d\mathbf{W} \\ &= \underbrace{\frac{\alpha_t}{\alpha_s}\mathbf{X}_s - \alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda} \epsilon_\lambda d\lambda + \int_{\lambda_s}^{\lambda_t} g(\lambda) d\mathbf{W}}_{\mathcal{H}(t, s, \mathbf{X}_s, \mathbf{X}_T)} \\ &\quad \underbrace{-\alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda} \delta(\mathbf{X}_\lambda, \lambda) d\lambda}_{\mathcal{Z}}.\end{aligned} \tag{31}$$

We can identify the first part, $\mathcal{H}$, as the ideal linear solver. By substituting the definition of the ground-truth noise, $\epsilon_\lambda = (\mathbf{X}_\lambda - \alpha_\lambda \mathbf{X}_T(\mathbf{X}_\lambda))/\sigma_\lambda$, this term becomes:

$$\begin{aligned}\mathcal{H}(t, s, \mathbf{X}_s, \mathbf{X}_T) = \frac{\alpha_t}{\alpha_s}\mathbf{X}_s - \alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda} \frac{\mathbf{X}_\lambda - \alpha_\lambda \mathbf{X}_T(\mathbf{X}_\lambda)}{\sigma_\lambda} d\lambda \\ + \int_{\lambda_s}^{\lambda_t} g(\lambda) d\mathbf{W},\end{aligned} \tag{32}$$

which is the linear solver structure presented in the proposition.

The second part, $\mathcal{Z}$, represents the cumulative error introduced by the non-ideal denoising model:

$$\mathcal{Z} = -\alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda} \delta(\mathbf{X}_\lambda, \lambda) d\lambda$$

We can bound the norm of this error term using the triangle inequality for integrals and Assumption 1:

$$\begin{aligned}\|\mathcal{Z}\| &= \left\| -\alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda} \delta(\mathbf{X}_\lambda, \lambda) d\lambda \right\| \\ &\leqslant \alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda} \|\delta(\mathbf{X}_\lambda, \lambda)\| d\lambda \\ &\leqslant \alpha_t \zeta \int_{\lambda_s}^{\lambda_t} e^{-\lambda} d\lambda \\ &= \alpha_t \zeta \left[-e^{-\lambda}\right]_{\lambda_s}^{\lambda_t} \\ &= \alpha_t \zeta \left(-e^{-\lambda_t} - (-e^{-\lambda_s})\right) \\ &= \alpha_t \zeta \left(e^{-\lambda_s} - e^{-\lambda_t}\right).\end{aligned} \tag{33}$$

By substituting the definition $e^{-\lambda} = \sigma/\alpha$, we get the final bound:

$$\|\mathcal{Z}\| \leqslant \alpha_t \zeta \left(\frac{\sigma_s}{\alpha_s} - \frac{\sigma_t}{\alpha_t}\right) = \zeta \sigma_t \left(\frac{\alpha_t \sigma_s}{\alpha_s \sigma_t} - 1\right). \tag{34}$$

B. Analysis of Convergence and Step Size

To demonstrate that the term $\left(\frac{\sigma_s}{\alpha_s} - \frac{\sigma_t}{\alpha_t}\right)$ is negligible in practice, we rely on the necessary conditions for high-quality sampling in diffusion probabilistic models.

First, as established in [9] (Theorem 2), the convergence of the sampling process is strictly dependent on the discretization step size $h = s - t$. Specifically, the total variation distance

between the sampled distribution and the true distribution is strictly bounded only when the maximum step size satisfies:

$$h \leqslant 1. \tag{35}$$

If h is to exceed 1, the local linearization errors inherent in the discretization (e.g., Euler-Maruyama or DPM-Solver) would dominate, rendering the solver mathematically unstable regardless of the model's accuracy. Therefore, the practical deployment of a solver operates within the regime $h \in (0, 1]$.

Second, in standard large-scale diffusion models (e.g., Stable Diffusion, IDDPM), the continuous forward process is strictly defined over a normalized time interval $t \in [0, 1]$.

- For high-fidelity generation, it is standard practice to use $N \in [20, 50]$ discretization steps with advanced solvers (e.g., DPM-Solver, UniPC). The average step size corresponds to $h \approx \frac{1}{N}$, resulting in $h \in [0.02, 0.05]$.
- Even in extreme few-step sampling scenarios (e.g., LCM methods with $N \approx 4$), the step size $h \approx 0.25$ remains well within the theoretical convergence bound of 1.

Consequently, the operational regime $h \in (0, 1)$ covers the vast majority of practical use cases.

Under this condition (h is small), we apply the Mean Value Theorem to the noise-to-signal ratio function $f(\tau) = \frac{\sigma_\tau}{\alpha_\tau}$. There exists a $\xi \in [s, t]$ such that:

$$\frac{\sigma_s}{\alpha_s} - \frac{\sigma_t}{\alpha_t} = f(s) - f(t) = f'(\xi) \cdot (s - t). \tag{36}$$

Taking the absolute value and substituting $h = |s - t|$:

$$\left| \frac{\sigma_s}{\alpha_s} - \frac{\sigma_t}{\alpha_t} \right| \approx \left| \frac{d}{dt} \left(\frac{\sigma_t}{\alpha_t} \right) \right| h. \tag{37}$$

Since the noise schedules σ_t and α_t are smooth monotonic functions in practical SDEs, their derivative is bounded. Consequently, as $h \to 0$, the difference term approaches 0 linearly. Thus, the error bound in Eq. (34) is minimized solely by enforcing the standard high-quality sampling constraint $h \leqslant 1$.

C. Derivation of the Specific Error Bounds

Following the fundamental error bound derived in Eq. (34):

$$\|\mathcal{Z}_s\| \leqslant \alpha_t \zeta \left(\frac{\sigma_s}{\alpha_s} - \frac{\sigma_t}{\alpha_t} \right) \tag{38}$$

we substitute the specific parameters for α_t (signal scaling) and σ_t (noise schedule) from Table 1 of the EDM framework. This leads to Table XVI.

Appendix B
Proof of Proposition 2

Proof. Let $\{\mathbf{X}_{t_n}\}_{n=0}^N$ be the exact trajectory generated by the sequential integral solver. From Proposition 1, the step transition is given by:

$$\mathbf{X}_{t_{n+1}} = \mathcal{H}(t_{n+1}, t_n, \mathbf{X}_{t_n}, \mathbf{X}_T) + \mathcal{Z}_n, \tag{39}$$

where $\mathbf{X}_T$ is the ground-truth data and $\|\mathcal{Z}_n\| \approx 0$ is the linearization error. Let $\{\hat{\mathbf{X}}_{t_n}\}_{n=0}^N$ be the solution to the ϱ-nonlinear system with the update rule:

$$\hat{\mathbf{X}}_{t_{n+1}} = \mathcal{H}(t_{n+1}, t_n, \hat{\mathbf{X}}_{t_n}, \mathbf{E}_{t_n}). \tag{40}$$

Here, $\mathbf{E}_{t_n}$ denotes the estimated clean sample. Based on Definition 2, this estimator is explicitly defined as:

$$\mathbf{E}_{t_n} = \begin{cases} \hat{\mathbf{X}}_{t_N}(\hat{\mathbf{X}}_{t_n}, \theta) & \text{if } n < \varrho, \\ \hat{\mathbf{X}}_{t_N}(\hat{\mathbf{X}}_{t_{\varrho-1}}, \theta) & \text{if } n \geqslant \varrho. \end{cases} \tag{41}$$

We define the cumulative state error as $\boldsymbol{\Delta}_{t_n} = \hat{\mathbf{X}}_{t_n} - \mathbf{X}_{t_n}$ and the clean sample estimation error as $\boldsymbol{\xi}_n = \mathbf{E}_{t_n} - \mathbf{X}_T$. To derive the error recurrence, we subtract Eq. (39) from Eq. (40). Using the integral definition of $\mathcal{H}$ from Eq. (7), we obtain:

$$\boldsymbol{\Delta}_{t_{n+1}} = \frac{\alpha_{t_{n+1}}}{\alpha_{t_n}} \boldsymbol{\Delta}_{t_n} - \alpha_{t_{n+1}} \int_{\lambda_{t_n}}^{\lambda_{t_{n+1}}} e^{-\lambda} \mathcal{I}_{\lambda,n} \, d\lambda - \mathcal{Z}_n, \tag{42}$$

where the integrand is

$$\mathcal{I}_{\lambda,n} = \frac{\hat{\mathbf{X}}_\lambda - \alpha_\lambda \mathbf{E}_{t_n}}{\alpha_\lambda} - \frac{\mathbf{X}_\lambda - \alpha_\lambda \mathbf{X}_T}{\alpha_\lambda}. \tag{43}$$

Using the Log-SNR property that $e^{-\lambda} = \sigma_\lambda / \alpha_\lambda$, this integrand simplifies as follows:

$$\begin{aligned} \mathcal{I}_{\lambda,n} &= \frac{\sigma_\lambda}{\alpha_\lambda} \left(\frac{\hat{\mathbf{X}}_\lambda - \mathbf{X}_\lambda}{\sigma_\lambda} - \frac{\alpha_\lambda (\mathbf{E}_{t_n} - \mathbf{X}_T)}{\sigma_\lambda} \right) \\ &= \frac{\boldsymbol{\Delta}_\lambda}{\alpha_\lambda} - \boldsymbol{\xi}_n. \end{aligned} \tag{44}$$

To evaluate the intermediate error $\boldsymbol{\Delta}_\lambda$ inside the integral, we invoke the **Superposition Principle** for linear differential equations [5], [7]. The total error $\boldsymbol{\Delta}_\lambda$ can be rigorously decomposed into the sum of the *Zero-Input Response* (Homogeneous) and the *Zero-State Response* (Forced):

- **Homogeneous Response**: This component represents the natural evolution of the initial state error $\boldsymbol{\Delta}_{t_n}$ through the system dynamics, independent of the external estimation error. Due to the signal scaling inherent in the diffusion process $\mathcal{H}$, this is given by:

$$\boldsymbol{\Delta}_\lambda^{\text{homo}} = \frac{\alpha_\lambda}{\alpha_{t_n}} \boldsymbol{\Delta}_{t_n}. \tag{45}$$

- **Forced Response**: This component captures the error accumulated over the interval $[\lambda_{t_n}, \lambda]$ solely due to the input source $\boldsymbol{\xi}_n$. Based on the integral structure of the linear solver in Eq. (7), the contribution accumulates as:

$$\boldsymbol{\Delta}_\lambda^{\text{force}} = \alpha_\lambda \left[\int_{\lambda_{t_n}}^{\lambda} e^{-\tau} \frac{\alpha_\tau}{\sigma_\tau} d\tau \right] \boldsymbol{\xi}_n. \tag{46}$$

Substituting $e^{-\tau} = \sigma_\tau / \alpha_\tau$, the integrand simplifies perfectly to unity:

$$\boldsymbol{\Delta}_\lambda^{\text{force}} = \alpha_\lambda \left[\int_{\lambda_{t_n}}^{\lambda} 1 d\tau \right] \boldsymbol{\xi}_n = \alpha_\lambda (\lambda - \lambda_{t_n}) \boldsymbol{\xi}_n. \tag{47}$$

TABLE XVI
PRACTICAL ERROR BOUNDS FOR DIFFUSION FRAMEWORKS DEFINED IN THE EDM DESIGN SPACE. PARAMETERS ARE MAPPED FROM TABLE 1 OF [22] WHERE $\beta_{\min} = 0.1, \beta_d = 19.9$,

Method	Schedule Settings from Table 1	Substitution	Final Error Bound
EDM [22]	$\alpha_t = 1$ $\sigma_t = \alpha_t \cdot t$	$\frac{\sigma_t}{\alpha_t} = t$	$\|\mathcal{Z}_s\| \leqslant \zeta(s-t) = \zeta h$
VE [50]	$\alpha_t = 1$ $\sigma_t = \alpha_t \cdot \sqrt{t}$	$\frac{\sigma_t}{\alpha_t} = \sqrt{t}$	$\|\mathcal{Z}_s\| \leqslant \zeta(\sqrt{s} - \sqrt{t})$
VP [50]	$\alpha_t = 1/\sqrt{e^{\frac{1}{2}\beta_d t^2 + \beta_{\min} t}}$ $\sigma_t = \alpha_t \cdot \sqrt{e^{\frac{1}{2}\beta_d t^2 + \beta_{\min} t} - 1}$	$\frac{\sigma_t}{\alpha_t} = \sqrt{e^{\frac{1}{2}\beta_d t^2 + \beta_{\min} t} - 1}$	$\|\mathcal{Z}_s\| \leqslant \zeta \alpha_t \left(\frac{\sigma_s}{\alpha_s} - \frac{\sigma_t}{\alpha_t} \right)$

Combining these components via superposition, we obtain the explicit expression for the intermediate error:

$$\boldsymbol{\Delta}_\lambda = \underbrace{\frac{\alpha_\lambda}{\alpha_{t_n}} \boldsymbol{\Delta}_{t_n}}_{\text{Homogeneous Response}} + \underbrace{\alpha_\lambda (\lambda - \lambda_{t_n}) \boldsymbol{\xi}_n}_{\text{Forced Response}}. \tag{48}$$

We substitute this expression for $\boldsymbol{\Delta}_\lambda$ into the term $\frac{\boldsymbol{\Delta}_\lambda}{\alpha_\lambda}$. This leads to an algebraic cancellation of the time-dependent coefficient α_λ in the homogeneous term:

$$\begin{aligned} \frac{\boldsymbol{\Delta}_\lambda}{\alpha_\lambda} &= \frac{1}{\alpha_\lambda} \left(\frac{\alpha_\lambda}{\alpha_{t_n}} \boldsymbol{\Delta}_{t_n} + \alpha_\lambda (\lambda - \lambda_{t_n}) \boldsymbol{\xi}_n \right) \\ &= \frac{\boldsymbol{\Delta}_{t_n}}{\alpha_{t_n}} + (\lambda - \lambda_{t_n}) \boldsymbol{\xi}_n. \end{aligned} \tag{49}$$

Now we can analytically evaluate the integral in Eq. (42):

$$\begin{aligned} &\int_{\lambda_{t_n}}^{\lambda_{t_{n+1}}} \left(\frac{\boldsymbol{\Delta}_\lambda}{\alpha_\lambda} - \boldsymbol{\xi}_n \right) d\lambda \\ &= \int_{\lambda_{t_n}}^{\lambda_{t_{n+1}}} \left(\frac{\boldsymbol{\Delta}_{t_n}}{\alpha_{t_n}} + (\lambda - \lambda_{t_n}) \boldsymbol{\xi}_n - \boldsymbol{\xi}_n \right) d\lambda \\ &= \frac{\Delta\lambda_n}{\alpha_{t_n}} \boldsymbol{\Delta}_{t_n} + \left[\frac{(\lambda - \lambda_{t_n})^2}{2} - (\lambda - \lambda_{t_n}) \right]_{\lambda_{t_n}}^{\lambda_{t_{n+1}}} \boldsymbol{\xi}_n \\ &= \frac{\Delta\lambda_n}{\alpha_{t_n}} \boldsymbol{\Delta}_{t_n} + \left(\frac{(\Delta\lambda_n)^2}{2} - \Delta\lambda_n \right) \boldsymbol{\xi}_n, \end{aligned} \tag{50}$$

where $\Delta\lambda_n = \lambda_{t_{n+1}} - \lambda_{t_n}$.

Plugging this integral result back into the expression for $\boldsymbol{\Delta}_{t_{n+1}}$ and grouping the terms by error source:

$$\begin{aligned} \boldsymbol{\Delta}_{t_{n+1}} = &\underbrace{\frac{\alpha_{t_{n+1}}}{\alpha_{t_n}} \left(1 - \Delta\lambda_n \right)}_{\mathcal{A}_n} \boldsymbol{\Delta}_{t_n} \\ &+ \underbrace{\alpha_{t_{n+1}} \Delta\lambda_n \left(1 - \frac{\Delta\lambda_n}{2} \right)}_{\mathcal{B}_n} \boldsymbol{\xi}_n \\ &- \mathcal{Z}_n. \end{aligned} \tag{51}$$

Analysis of Coefficients: The convergence of the solver relies on the discretization step size $h = |t_{n+1} - t_n|$. As established in Theorem 2 of [9], high-quality generation strictly requires $h \leqslant 1$. Under this condition, we analyze the bounds of $\mathcal{A}_n$ and $\mathcal{B}_n$.

First, the coefficient $\mathcal{A}_n$ determines the propagation of historical errors. We evaluate its magnitude $|\mathcal{A}_n|$ across mainstream diffusion configurations:

- **EDM and VE-SDE Schedules ($\alpha_t \equiv 1$):** The signal scale ratio is unity ($\frac{\alpha_{t_{n+1}}}{\alpha_{t_n}} = 1$). The coefficient simplifies to:
$$|\mathcal{A}_n| = |1 - \Delta\lambda_n|. \tag{52}$$
Since $\Delta\lambda_n$ is a small positive value determined by the small step size, we have $|\mathcal{A}_n| < 1$. This indicates strictly contractive dynamics, where historical errors naturally decay over time.
- **VP-SDE Schedule (α_t varies):** Although α_t increases during the reverse process, the decay term $(1 - \Delta\lambda_n)$ counteracts this growth. Using the first-order approximation $1 - \Delta\lambda_n \approx e^{-\Delta\lambda_n}$ and the definition $e^{-\lambda} = \sigma/\alpha$:
$$\begin{aligned} \mathcal{A}_n &\approx \frac{\alpha_{t_{n+1}}}{\alpha_{t_n}} e^{-(\lambda_{t_{n+1}} - \lambda_{t_n})} \\ &= \frac{\alpha_{t_{n+1}}}{\alpha_{t_n}} \frac{\sigma_{t_{n+1}}/\alpha_{t_{n+1}}}{\sigma_{t_n}/\alpha_{t_n}} \\ &= \frac{\sigma_{t_{n+1}}}{\sigma_{t_n}}. \end{aligned} \tag{53}$$
Since the noise level σ_t strictly decreases in the generative direction, the ratio satisfies $\frac{\sigma_{t_{n+1}}}{\sigma_{t_n}} < 1$. Thus, $|\mathcal{A}_n| < 1$ holds for VP schedules as well.

The error propagation is non-expansive ($|\mathcal{A}_n| \leqslant 1$), ensuring that prior errors $\boldsymbol{\Delta}_{t_n}$ do not explode during the sequential process.

Second, the coefficient $\mathcal{B}_n$ governs the sensitivity of the trajectory to the non-ideal estimator error $\boldsymbol{\xi}_n$. Using the Mean Value Theorem, the log-SNR increment is proportional to the step size: $\Delta\lambda_n \approx |\dot{\lambda}| h$. Since the noise schedule derivatives are bounded, a small step size $h \leqslant 1$ implies $\Delta\lambda_n \ll 1$.

We bound $|\mathcal{B}_n|$ as follows:

- Since $|\alpha_{t_{n+1}}| \leqslant 1$ (strictly 1 for EDM/VE, $\leqslant 1$ for VP) and $\Delta\lambda_n > 0$:
$$|\mathcal{B}_n| \leqslant \Delta\lambda_n \left| 1 - \frac{\Delta\lambda_n}{2} \right|. \tag{54}$$

- In the practical regime where $h \leqslant 1$, we maintain $\Delta\lambda_n < 2$, ensuring the term $(1 - \Delta\lambda_n/2)$ is strictly less than 1. Thus:
$$|\mathcal{B}_n| < \Delta\lambda_n \approx \mathcal{O}(h). \tag{55}$$

Even in few-step generation scenarios (e.g., $N = 4, h \approx 0.25$), $|\mathcal{B}_n|$ remains a small fractional scalar. As $h \to 0$, $|\mathcal{B}_n|$ approaches zero linearly. This proves that the solver *suppresses* the injection of the estimation error $\boldsymbol{\xi}_n$ rather than accumulating it linearly.

Accumulated bound. Combining the established bounds with Eq. (51), the per-step error inequality is

$$\|\boldsymbol{\Delta}_{t_{n+1}}\| \leqslant \underbrace{|\mathcal{A}_n|}_{\leqslant 1} \|\boldsymbol{\Delta}_{t_n}\| + \underbrace{|\mathcal{B}_n|}_{\mathcal{O}(h)} \|\boldsymbol{\xi}_n\| + \underbrace{\|\mathcal{Z}_n\|}_{\zeta\,\mathcal{O}(h)}. \tag{56}$$

Starting from the exact initial condition $\boldsymbol{\Delta}_{t_0} = \mathbf{0}$, unrolling the recurrence over the N discretization steps yields the *accumulated error bound*:

$$\begin{aligned}\|\boldsymbol{\Delta}_{t_N}\| &\leqslant \sum_{n=0}^{N-1} \Big(\prod_{j=n+1}^{N-1} |\mathcal{A}_j| \Big) \big(|\mathcal{B}_n| \|\boldsymbol{\xi}_n\| + \|\mathcal{Z}_n\| \big) \\ &\leqslant \sum_{n=0}^{N-1} \big(|\mathcal{B}_n| \|\boldsymbol{\xi}_n\| + \|\mathcal{Z}_n\| \big) \leqslant N\,\mathcal{O}(h) \big(\|\boldsymbol{\xi}\|_{\max} + \zeta \big),\end{aligned} \tag{57}$$

where the second inequality uses $|\mathcal{A}_j| \leqslant 1$ and the third substitutes $|\mathcal{B}_n| = \mathcal{O}(h)$ and $\|\mathcal{Z}_n\| = \zeta\,\mathcal{O}(h)$ from Proposition 1. With $h = \mathcal{O}(1/N)$, this reduces to $\mathcal{O}(\|\boldsymbol{\xi}\|_{\max} + \zeta)$; for a 50-step schedule with $h = 1/50$ and $\zeta \sim 10^{-2}$ the bound is on the order of 10^{-2}, negligible relative to an $O(1)$ signal scale.

Dependence on ϱ, SOR, and adaptive stopping. The bound is *independent of ϱ in structure*: ϱ only changes the source of the estimator $\mathbf{E}_{t_n}$ (and hence the magnitudes $\|\boldsymbol{\xi}_n\|$), not the propagation/sensitivity coefficients $\mathcal{A}_n$ and $\mathcal{B}_n$. The successive-over-relaxation update of ParaLin++ shares the same fixed point and is stable for $0 < \omega < 2/(1 + L)$ (Proposition 4), so it obeys the same per-step recurrence and hence the same accumulated bound; SOR only accelerates the local contraction. The adaptive stopping criterion (Section IV-B4) replaces the exact fixed point by a δ-neighborhood, adding an additional $O(\delta)$ truncation, so the realized deviation satisfies

$$\|\boldsymbol{\Delta}_{t_N}^{(\text{ParaLin++})}\| \leqslant N\,\mathcal{O}(h) \big(\|\boldsymbol{\xi}\|_{\max} + \zeta \big) + \mathcal{O}(\delta). \tag{58}$$

Conclusion. Hence the ϱ-nonlinear surrogate trajectory $\{\hat{\mathbf{X}}_{t_n}\}$ deviates from the exact sequential trajectory $\{\mathbf{X}_{t_n}\}$ by a bounded, small quantity, *not* by zero. ParaLin converges to the surrogate system rather than exactly to the nonlinear sequential sampler; this is the gap bounded in Proposition 2 and the corrected convergence target formalized in Proposition 3. □

APPENDIX C
PROOF OF CONVERGENCE IN PROPOSITION 3

Let $\{\hat{\mathbf{X}}^\star_{t_n}\}_{n=0}^N$ denote the unique solution of the ϱ-nonlinear surrogate system in Eq. (11) (the partially linear surrogate trajectory), which by Proposition 2 deviates from the exact sequential trajectory $\{\mathbf{X}_{t_n}\}$ by the accumulated error bound Eq. (57). We show that the iteration Eq. (18) converges to $\{\hat{\mathbf{X}}^\star_{t_n}\}$, *not* to $\{\mathbf{X}_{t_n}\}$.

Assume by induction that for iteration k, the first n components have converged to the surrogate, $\{\hat{\mathbf{X}}^{(k)}_{t_i}\}_{i=0}^{n-1} = \{\hat{\mathbf{X}}^\star_{t_i}\}_{i=0}^{n-1}$ (and, when $n \geqslant \varrho$, also $\hat{\mathbf{X}}^{(k)}_{t_{\varrho-1}} = \hat{\mathbf{X}}^\star_{t_{\varrho-1}}$, since $\varrho - 1 < n$). For the next component $\hat{\mathbf{X}}^{(k+1)}_{t_n}$, define the iterate and surrogate maps

$$\begin{aligned}\mathcal{B}_n^{(k)} &= \mathcal{H}\Big(t_n, t_{n-1}, \hat{\mathbf{X}}^{(k)}_{t_{n-1}}, \mathbf{E}^{(k)}_{t_{n-1}}\Big), \\ \mathcal{B}_n^{\star} &= \mathcal{H}\Big(t_n, t_{n-1}, \hat{\mathbf{X}}^{\star}_{t_{n-1}}, \mathbf{E}^{\star}_{t_{n-1}}\Big),\end{aligned} \tag{59}$$

where $\mathbf{E}^{(k)}_{t_{n-1}} = \hat{\mathbf{X}}_{t_N}(\hat{\mathbf{X}}^{(k)}_{t_{n-1}}, \theta)$ for $n < \varrho$ and $\mathbf{E}^{(k)}_{t_{n-1}} = \hat{\mathbf{X}}_{t_N}(\hat{\mathbf{X}}^{(k)}_{t_{\varrho-1}}, \theta)$ for $n \geqslant \varrho$ (Definition 2), and $\mathbf{E}^{\star}_{t_{n-1}}$ is its surrogate analogue. By the surrogate equation Eq. (11), $\hat{\mathbf{X}}^\star_{t_n} = \mathcal{B}^\star_n$. Therefore

$$\begin{aligned}\hat{\mathbf{X}}^{(k+1)}_{t_n} &= \hat{\mathbf{X}}^{(k)}_{t_n} - \mathcal{G}^{(k)} \mathcal{R}^{(k)}_{t_n} = \hat{\mathbf{X}}^{(k)}_{t_n} - \mathcal{G}^{(k)} \big(\hat{\mathbf{X}}^{(k)}_{t_n} - \mathcal{B}^{(k)}_n \big) \\ &\overset{(a)}{=} \hat{\mathbf{X}}^{(k)}_{t_n} - \mathcal{G}^{(k)} \big(\hat{\mathbf{X}}^{(k)}_{t_n} - \mathcal{B}^{\star}_n \big) \\ &\overset{(b)}{=} \mathcal{B}^{\star}_n = \hat{\mathbf{X}}^{\star}_{t_n},\end{aligned} \tag{60}$$

with

- Eq. (a) from the induction hypothesis: $\hat{\mathbf{X}}^{(k)}_{t_{n-1}} = \hat{\mathbf{X}}^\star_{t_{n-1}}$ (and $\hat{\mathbf{X}}^{(k)}_{t_{\varrho-1}} = \hat{\mathbf{X}}^\star_{t_{\varrho-1}}$ for $n \geqslant \varrho$) gives $\mathcal{B}^{(k)}_n = \mathcal{B}^\star_n$;
- Eq. (b) since $\mathcal{G}^{(k)} = \mathbf{I}$ or $\mathcal{G}^{(k)} = \mathcal{J}^{-1} = \mathbf{I}$, where $\mathcal{J} = \partial(\hat{\mathbf{X}}^{(k)}_{t_n} - \mathcal{B}^\star_n)/\partial \hat{\mathbf{X}}^{(k)}_{t_n} = \mathbf{I}$, because $\mathcal{B}^\star_n$ depends on $\hat{\mathbf{X}}^{(k)}_{t_n}$ only through the already-converged predecessors ($\hat{\mathbf{X}}^\star_{t_{n-1}}$ and, for $n \geqslant \varrho$, $\hat{\mathbf{X}}^\star_{t_{\varrho-1}}$), not through $\hat{\mathbf{X}}^{(k)}_{t_n}$ itself.

Thus each iteration advances the convergence front by at least one index, giving $K \leqslant N$ in the worst case (and $K \ll N$ in practice via immediate propagation of newly converged components).

Convergence target. The argument establishes convergence of the iterative solve to $\{\hat{\mathbf{X}}^\star_{t_n}\}$, i.e., to the ϱ-nonlinear surrogate, within $K \leqslant T$ iterations. It does *not* reach the exact sequential trajectory: $\{\hat{\mathbf{X}}^\star_{t_n}\} \neq \{\mathbf{X}_{t_n}\}$ in general, and the residual gap $\|\hat{\mathbf{X}}^\star_{t_n} - \mathbf{X}_{t_n}\|$ is the accumulated error bounded in Proposition 2 (Eq. Eq. (57)). The SOR update keeps this same fixed point (Proposition 4), and the adaptive stopping reintroduces only an $O(\delta)$ deviation.

APPENDIX D
PROOF FOR PROPOSITION 4

Proof. Let the iterative solver described by the immediate update strategy in Eq. (18) be represented by a mapping $\mathcal{F} : \mathbb{R}^{d\times N} \to \mathbb{R}^{d\times N}$, where d is the data dimension and N is the number of discretization steps. The update rule can be written as:

$$\hat{\mathbf{X}}^{(k+1)} = \mathcal{F}(\hat{\mathbf{X}}^{(k)}).$$

For the iterative method to converge, the mapping $\mathcal{F}$ must be a contraction in a neighborhood of the unique solution $\mathbf{X}^*$. This means there exists a constant $L < 1$ such that for any

two trajectories $\mathbf{X}$ and $\mathbf{Y}$ in this neighborhood, the following holds:

$$\|\mathcal{F}(\mathbf{X}) - \mathcal{F}(\mathbf{Y})\| \leqslant L\|\mathbf{X} - \mathbf{Y}\|.$$

The SOR update modifies this iteration. Let the operator for the SOR-enhanced update be T_{SOR}. From Eq. (21), we can write the SOR update as:

$$\hat{\mathbf{X}}^{(k+1)} = T_{SOR}(\hat{\mathbf{X}}^{(k)}) = (1-\omega)\hat{\mathbf{X}}^{(k)} + \omega\mathcal{F}(\hat{\mathbf{X}}^{(k)}).$$

To analyze the convergence of the SOR method, we must show that T_{SOR} is also a contraction mapping. We analyze its contraction factor, L_{SOR}:

$$\begin{aligned}\|T_{SOR}(\mathbf{X}) - T_{SOR}(\mathbf{Y})\| = \big\|&(1-\omega)(\mathbf{X}-\mathbf{Y}) \\ &+ \omega(\mathcal{F}(\mathbf{X}) - \mathcal{F}(\mathbf{Y}))\big\|.\end{aligned}$$

Using the triangle inequality, we can bound this expression:

$$\begin{aligned}\|T_{SOR}(\mathbf{X}) - T_{SOR}(\mathbf{Y})\| &\leqslant |1-\omega|\|\mathbf{X}-\mathbf{Y}\| \\ &\quad + \omega\|\mathcal{F}(\mathbf{X}) - \mathcal{F}(\mathbf{Y})\| \\ &= \big(|1-\omega| + \omega L\big)\|\mathbf{X}-\mathbf{Y}\|.\end{aligned}$$

Since $\mathcal{F}$ is a contraction with factor L, we have:

$$\|T_{SOR}(\mathbf{X}) - T_{SOR}(\mathbf{Y})\| \leqslant (|1-\omega| + \omega L)\|\mathbf{X}-\mathbf{Y}\|.$$

Thus, the contraction factor for the SOR iteration is $L_{SOR} = |1-\omega| + \omega L$. For convergence, we require $L_{SOR} < 1$. We analyze this condition in two cases based on the value of ω.

Case 1: Under-relaxation ($0 < \omega \leqslant 1$) In this range, $|1-\omega| = 1-\omega$. The contraction factor becomes:

$$L_{SOR} = (1-\omega) + \omega L = 1 - \omega(1-L).$$

Since the base method is assumed to converge, we have $L < 1$, which implies $1 - L > 0$. As $\omega > 0$, the term $\omega(1-L)$ is strictly positive. Therefore, $L_{SOR} = 1 - (\text{positive value}) < 1$. Convergence is guaranteed for any $\omega \in (0, 1]$.

Case 2: Over-relaxation ($\omega > 1$) In this range, $|1-\omega| = \omega - 1$. The contraction factor becomes:

$$L_{SOR} = (\omega - 1) + \omega L = \omega(1+L) - 1.$$

For convergence, we require $L_{SOR} < 1$:

$$\omega(1+L) - 1 < 1 \implies \omega(1+L) < 2 \implies \omega < \frac{2}{1+L}.$$

Combining the condition $\omega > 1$ with this derived upper bound, convergence for over-relaxation is guaranteed when $1 < \omega < \frac{2}{1+L}$.

Combining both cases, the SOR-enhanced solver is guaranteed to converge if the relaxation factor ω is chosen in the range:

$$0 < \omega < \frac{2}{1+L}.$$

Since $0 \leqslant L < 1$, the upper bound $2/(1+L)$ is always in the range $(1, 2]$. This analysis provides the effective range for ω and shows that over-relaxation can maintain convergence under a computable condition related to the contractiveness of the underlying solver. □